

**BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT TP. HCM
KHOA CƠ KHÍ CHẾ TẠO MÁY**



HCM-UTE

MÔN HỌC: HỌC SÂU

BÁO CÁO

**XÂY DỰNG HỆ THỐNG MẪU NHẬN DẠNG
HÀNH ĐỘNG TỪ VIDEO SỬ DỤNG MẠNG
NƠ-RON TÍCH CHẬP 3 CHIỀU (3D CNN)**

GVHD: TS. Vũ Quang Huy

SVTH: MSSV

Lương Minh Nhật 23134042

Mã lớp: 251DELE337829_01

Tp. Hồ Chí Minh, tháng 1 năm 2026

LỜI CẢM ƠN

Trong suốt quá trình học tập và nghiên cứu học phần Học sâu (Deep Learning), việc hoàn thành báo cáo này không chỉ là kết quả của sự nỗ lực cá nhân, mà còn là thành quả của sự quan tâm, chỉ dẫn và hỗ trợ tận tình từ quý Thầy Cô và môi trường học thuật của Khoa Cơ khí Chế tạo máy – Trường Đại học Công nghệ Kỹ thuật TP.HCM. Nhân dịp hoàn thành báo cáo, em xin được bày tỏ lòng tri ân sâu sắc đến tất cả những cá nhân và tập thể đã đồng hành cùng em trong suốt quá trình học tập và thực hiện đề tài.

Trước hết, em xin trân trọng gửi lời cảm ơn chân thành và sâu sắc nhất đến TS. Vũ Quang Huy, giảng viên trực tiếp giảng dạy và hướng dẫn học phần. Với kiến thức chuyên môn sâu rộng, phương pháp giảng dạy khoa học và tinh thần trách nhiệm cao, Thầy đã tận tình hướng dẫn, giải đáp thắc mắc và định hướng tư duy cho em trong suốt quá trình tiếp cận các kiến thức phức tạp của lĩnh vực Trí tuệ nhân tạo. Những kiến thức nền tảng về mạng nơ-ron nhân tạo, thị giác máy tính và kỹ thuật xử lý dữ liệu mà Thầy truyền đạt không chỉ giúp em hoàn thành tốt báo cáo, mà còn góp phần hình thành tư duy giải quyết vấn đề và phương pháp nghiên cứu khoa học nghiêm túc cho các dự án sau này.

Mặc dù đã có nhiều cố gắng trong quá trình thực hiện, song do giới hạn về thời gian, kinh nghiệm và kiến thức chuyên môn, bài báo cáo không tránh khỏi những thiếu sót nhất định. Em kính mong nhận được sự góp ý, nhận xét và chỉ dẫn thêm từ Thầy để có thể rút kinh nghiệm, tiếp tục hoàn thiện bản thân và nâng cao chất lượng học tập, nghiên cứu trong tương lai.

Tp. Hồ Chí Minh, tháng 1 năm 2026

Sinh viên thực hiện

Lương Minh Nhật

MỤC LỤC

CHƯƠNG 1. GIỚI THIỆU TỔNG QUAN	1
1.1. Đặt vấn đề	1
1.2. Lý do chọn đề tài	2
1.3. Mục tiêu của đề tài	3
1.4. Đối tượng và phạm vi nghiên cứu	3
1.4.1. Đối tượng nghiên cứu	3
1.4.2. Phạm vi nghiên cứu	4
1.5. Bố cục báo cáo	4
CHƯƠNG 2. CƠ SỞ LÝ THUYẾT	6
2.1. Tổng quan về bài toán nhận dạng hành động (Human Action Recognition)	6
2.1.1. Định nghĩa bài toán	6
2.1.2. Hai thành phần đặc trưng của video	6
2.1.3. Các thách thức chính	6
2.2. Mạng nơ-ron tích chập (Convolutional Neural Network – CNN)	7
2.2.1. Giới thiệu chung	7
2.2.2. Các thành phần kiến trúc chính	7
2.2.3. Cơ chế hoạt động trên dữ liệu ảnh	8
2.3. Hạn chế của CNN 2D trong xử lý video	9
2.3.1. Vấn đề xử lý độc lập từng khung hình	9
2.3.2. Mất mát thông tin ngữ nghĩa toàn cục	10
2.4. Mạng nơ-ron tích chập 3 chiều (3D CNN)	10
2.4.1. Khái niệm và nguyên lý hoạt động	10
2.4.2. Mô hình toán học	11
2.4.3. Kiến trúc C3D tham khảo	12
2.4.4. So sánh tổng hợp giữa 2D CNN và 3D CNN	12
2.5. Các thành phần hỗ trợ trong huấn luyện mô hình	13
2.5.1. Hàm kích hoạt đầu ra (Output Activation)	13
2.5.2. Hàm mất mát (Loss Function)	13
2.5.3. Thuật toán tối ưu hóa (Optimizer)	14
2.6. Đánh giá ưu và nhược điểm của mô hình đề xuất	14

CHƯƠNG 3. DỮ LIỆU HUẤN LUYỆN	15
3.1. Giới thiệu chung về dữ liệu huấn luyện	15
3.2. Bộ dữ liệu UCF50	15
3.3. Lựa chọn tập con dữ liệu cho hệ thống mẫu	17
3.4. Cấu trúc và định dạng dữ liệu	17
3.5. Đặc trưng đầu vào và đầu ra của mô hình	18
3.6. Các bước tiền xử lý dữ liệu	20
3.7. Phân chia tập dữ liệu huấn luyện và kiểm tra	21
3.8. Nhận xét và đánh giá tập dữ liệu	22
3.9. Kết luận chương	22
CHƯƠNG 4. LƯU ĐỒ THUẬT TOÁN	24
4.1. Tổng quan thuật toán huấn luyện	24
4.2. Lưu đồ thuật toán huấn luyện mô hình	25
4.3. Giải thích chi tiết các bước trong lưu đồ	26
CHƯƠNG 5. CODE CHƯƠNG TRÌNH	29
5.1. Môi trường cài đặt và thư viện sử dụng	29
5.2. Nguồn code tham khảo và các cải tiến chính	31
5.3. Mô tả các khối chương trình và giải thích code	33
5.4. So sánh chi tiết code gốc và code cải tiến	40
5.5. Minh chứng thực thi chương trình	41
CHƯƠNG 6. PHÂN TÍCH VÀ ĐÁNH GIÁ KẾT QUẢ	45
6.1. Các chỉ số đánh giá sử dụng (Evaluation Metrics)	45
6.2. Phân tích quá trình huấn luyện (Training Analysis)	47
6.2.1. Phân tích biến thiên của hàm mất mát (Loss)	47
6.2.2. Phân tích biến thiên của độ chính xác (Accuracy)	48
6.3. Đánh giá trên tập dữ liệu kiểm thử (Testing Evaluation)	49
6.3.1. Phân tích Ma trận nhầm lẫn (Confusion Matrix)	49
6.3.2. Phân tích các chỉ số chi tiết (Precision, Recall, F1-Score)	50
6.3.3. Đánh giá tốc độ suy diễn (Inference Speed)	51
6.4. So sánh hiệu quả với mô hình tham khảo (Benchmarking)	52
6.4.1. So sánh về các chỉ số định lượng	52

6.4.2. So sánh trực quan về Khoảng cách Tổng quát hóa (Generalization Gap)....	53
6.4.3. Phân tích nguyên nhân cải tiến	54
CHƯƠNG 7. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	55
7.1. Kết luận chung	55
7.2. Các hạn chế của đề tài.....	55
7.3. Hướng phát triển	56
TÀI LIỆU THAM KHẢO.....	57
PHỤ LỤC	59
PHỤ LỤC A: CẤU HÌNH MÔI TRƯỜNG VÀ THƯ VIỆN THỰC NGHIỆM	59
PHỤ LỤC B: CHI TIẾT VỀ TẬP DỮ LIỆU (DATASET)	60
PHỤ LỤC C: CHI TIẾT KIẾN TRÚC MÔ HÌNH VÀ MÃ NGUỒN.....	61
PHỤ LỤC D: HƯỚNG DẪN SỬ DỤNG VÀ CHẠY CHƯƠNG TRÌNH.....	62

CHƯƠNG 1. GIỚI THIỆU TỔNG QUAN

1.1. Đặt vấn đề

Trong kỷ nguyên công nghệ số và bùng nổ thông tin hiện nay, dữ liệu video đang gia tăng với tốc độ chóng mặt. Sự phổ biến của các thiết bị ghi hình như camera giám sát (CCTV), điện thoại thông minh, cùng với sự phát triển của các nền tảng chia sẻ video trực tuyến (YouTube, TikTok) đã tạo ra một kho dữ liệu khổng lồ chứa đựng thông tin phong phú về hoạt động của con người. Tuy nhiên, việc lưu trữ lượng dữ liệu này sẽ trở nên vô nghĩa nếu thiếu đi khả năng phân tích và thấu hiểu nội dung bên trong nó. Việc dựa vào con người để giám sát và phân loại thủ công lượng video này là bất khả thi về mặt thời gian và chi phí.

Xuất phát từ thực tế đó, bài toán Nhận dạng hành động con người (Human Action Recognition - HAR) đã trở thành một chủ đề nghiên cứu trọng tâm trong lĩnh vực Thị giác máy tính (Computer Vision). Mục tiêu của bài toán là xây dựng các hệ thống thông minh có khả năng tự động phát hiện và phân loại hành động của con người từ video, mở ra tiềm năng ứng dụng to lớn trong nhiều lĩnh vực như: giám sát an ninh thông minh, chăm sóc sức khỏe (phát hiện té ngã ở người cao tuổi), phân tích thể thao và tương tác người - máy.

Trong những năm gần đây, sự phát triển vượt bậc của Học sâu (Deep Learning), đặc biệt là Mạng nơ-ron tích chập (Convolutional Neural Networks - CNN), đã mang lại những thành tựu đột phá trong việc nhận dạng ảnh tĩnh. Tuy nhiên, việc áp dụng trực tiếp các kiến trúc 2D CNN truyền thống vào dữ liệu video gặp phải những hạn chế lớn. Video không chỉ là tập hợp của các hình ảnh tĩnh mà còn chứa đựng thông tin về sự vận động theo thời gian. Các mô hình 2D thường thất bại trong việc nắm bắt mối quan hệ ràng buộc giữa các khung hình liên tiếp, dẫn đến việc nhận diện sai các hành động có bối cảnh giống nhau nhưng diễn biến khác nhau.

Để giải quyết thách thức này, Mạng nơ-ron tích chập 3 chiều (3D CNN) được đề xuất như một giải pháp ưu việt, cho phép trích xuất đồng thời cả đặc trưng không gian (hình dáng, bối cảnh) và đặc trưng thời gian (chuyển động) trong một kiến trúc thống nhất. Nhận thấy tầm quan trọng và tính ứng dụng cao của kỹ thuật này, đề tài "Xây dựng hệ thống mẫu nhận dạng hành động từ video sử dụng Mạng nơ-ron tích chập 3 chiều (3D CNN)" được lựa chọn thực hiện nhằm nghiên cứu sâu về kiến trúc 3D CNN, đồng thời

xây dựng một hệ thống mẫu có khả năng nhận diện chính xác và hiệu quả các hành động cơ bản, khắc phục các nhược điểm của các phương pháp cũ.

1.2. Lý do chọn đề tài

Việc lựa chọn nghiên cứu và triển khai mô hình 3D CNN cho bài toán nhận dạng hành động xuất phát từ những nhu cầu cấp thiết về mặt thực tiễn và các hạn chế kỹ thuật của các phương pháp tiên nhiệm, cụ thể như sau:

Thứ nhất, khắc phục điểm yếu chí mạng của các mô hình 2D truyền thống. Các phương pháp nhận dạng hành động dựa trên 2D CNN thường xử lý video bằng cách tách rời từng khung hình hoặc sử dụng các kỹ thuật trích xuất đặc trưng thủ công phức tạp như Luồng quang học (Optical Flow). Cách tiếp cận này thường xuyên thất bại trong việc phân biệt các hành động có bối cảnh không gian tương đồng nhưng khác biệt về trình tự thực hiện (ví dụ: "ngồi xuống" và "đứng lên"). Do đó, việc chuyển sang một kiến trúc có khả năng học trực tiếp các đặc trưng động học từ dữ liệu thô là bước tiến bắt buộc.

Thứ hai, tính ưu việt trong kiến trúc của 3D CNN. Khác với mạng 2D, mạng nơ-ron tích chập 3 chiều sử dụng các bộ lọc khối (3D kernels) trượt trên cả ba chiều: chiều rộng, chiều cao và trục thời gian. Điều này cho phép mô hình mã hóa thông tin chuyển động (motion information) ngay trong quá trình tích chập, giúp bảo toàn tính liên kết của hành động. Đây là một trong những kiến trúc hiệu quả và trực tiếp nhất để khai thác đồng thời đặc trưng không gian – thời gian từ dữ liệu dạng khối (volumetric data) như video.

Thứ ba, nhu cầu kiểm chứng và tối ưu hóa hệ thống trong điều kiện thực tế. Mặc dù các kiến trúc 3D CNN đã được đề xuất và chứng minh hiệu quả về mặt lý thuyết, việc triển khai một hệ thống thực tế có khả năng cân bằng giữa độ chính xác và chi phí tính toán vẫn là một thách thức lớn, đặc biệt trong bối cảnh tài nguyên phần cứng hạn chế. Đề tài này không chỉ dừng lại ở việc ứng dụng mô hình 3D CNN cơ bản mà còn tập trung đánh giá và cải tiến hiệu năng thông qua một số kỹ thuật như Global Average Pooling và chiến lược lấy mẫu dữ liệu phù hợp. Mục tiêu là kiểm chứng tính khả thi của việc xây dựng một hệ thống nhận diện hành động có độ chính xác cao, đồng thời đạt được tốc độ xử lý thời gian thực ở mức suy diễn mô hình (inference-level real-time), hướng tới các ứng dụng phân tích hành động gần thời gian thực trong điều kiện phần cứng giới hạn.

Vì những lý do trên, đề tài này được thực hiện với mong muốn đóng góp một cái nhìn cụ thể, khoa học về quy trình xây dựng hệ thống nhận dạng hành động hiện đại, tạo tiền đề cho các ứng dụng giám sát thông minh trong tương lai.

1.3. Mục tiêu của đề tài

Mục tiêu tổng quát của đề tài là nghiên cứu, thiết kế và xây dựng thành công một hệ thống mẫu (prototype) có khả năng nhận dạng hành động con người từ video với độ chính xác cao và tốc độ xử lý nhanh, sử dụng kiến trúc mạng nơ-ron tích chập 3 chiều. Để đạt được mục đích này, đề tài tập trung giải quyết các mục tiêu cụ thể sau:

Về mặt lý thuyết: Nắm vững kiến thức nền tảng về Học sâu (Deep Learning) trong xử lý video. Phân tích sâu sắc cấu trúc và nguyên lý hoạt động của mạng 3D CNN, đồng thời làm rõ sự khác biệt và ưu thế của nó so với các kiến trúc 2D CNN truyền thống hay các mạng lai ghép (như ConvLSTM).

Về mặt thực nghiệm: Xây dựng hoàn chỉnh quy trình (pipeline) xử lý bài toán nhận dạng hành động, bao gồm các bước: thu thập và tiền xử lý dữ liệu video, thiết kế kiến trúc mạng, huấn luyện và kiểm thử mô hình. Đề tài sử dụng bộ dữ liệu chuẩn UCF50 (trích xuất tập con các hành động tiêu biểu) để làm cơ sở dữ liệu huấn luyện.

Về mặt cải tiến và tối ưu: Không chỉ dừng lại ở việc áp dụng mô hình cơ sở, đề tài hướng đến việc cải tiến hiệu năng thông qua các kỹ thuật hiện đại như: chiến lược lấy mẫu ngẫu nhiên liên tục (Random Continuous Sampling) để tăng cường dữ liệu, và sử dụng lớp Global Average Pooling để tối ưu hóa tham số mạng. Mục tiêu hướng đến cải thiện đáng kể hiệu năng so với mã nguồn tham khảo, thể hiện qua các chỉ số Accuracy, Loss và khả năng tổng quát hóa.

Về mặt đánh giá: Kiểm chứng độ tin cậy của mô hình thông qua các chỉ số định lượng khắt khe (Accuracy, Loss, Precision, Recall, F1-Score) và đánh giá khả năng ứng dụng thực tế thông qua việc đo đặc tốc độ suy diễn (Inference Speed) nhằm hướng tới khả năng xử lý thời gian thực ở mức suy diễn mô hình (inference-level real-time).

1.4. Đối tượng và phạm vi nghiên cứu

1.4.1. Đối tượng nghiên cứu

Đề tài tập trung nghiên cứu vào các đối tượng chính sau:

Mô hình học sâu: Trọng tâm là kiến trúc Mạng nơ-ron tích chập 3 chiều (3D CNN) và các kỹ thuật tối ưu hóa liên quan (Global Average Pooling, Dropout) để xử lý dữ liệu video.

Dữ liệu video hành động: Các đặc trưng thị giác (không gian) và đặc trưng chuyển động (thời gian) của con người trong video. Bộ dữ liệu được sử dụng để nghiên cứu và huấn luyện là bộ dữ liệu chuẩn UCF50.

1.4.2. Phạm vi nghiên cứu

Để đảm bảo tính khả thi trong khuôn khổ thời gian và tài nguyên của một báo cáo môn học, phạm vi nghiên cứu được giới hạn cụ thể như sau:

Phạm vi về dữ liệu: Do yêu cầu lớn về năng lực tính toán của mô hình 3D, đề tài không thực hiện trên toàn bộ 50 lớp hành động của bộ dữ liệu UCF50 mà lựa chọn một tập con gồm 4 lớp hành động tiêu biểu: WalkingWithDog (Dắt chó đi dạo), TaiChi (Thái cực quyền), Swing (Đu dây) và HorseRace (Đua ngựa). Các lớp này được chọn lọc để đại diện cho các kiểu chuyển động khác nhau (từ chậm đến nhanh, từ tĩnh tại đến di chuyển mạnh).

Phạm vi về kỹ thuật: Đề tài tập trung sâu vào phương pháp trích xuất đặc trưng trực tiếp từ chuỗi khung hình RGB (End-to-End Learning) bằng 3D CNN. Các phương pháp tiền xử lý phức tạp khác như trích xuất Luồng quang học (Optical Flow) hay trích xuất khung xương (Skeleton-based) nằm ngoài phạm vi nghiên cứu của báo cáo này.

Phạm vi về ứng dụng: Hệ thống được xây dựng dừng lại ở mức độ phần mềm mô phỏng (Prototype) chạy trên môi trường máy tính cá nhân và nền tảng đám mây (Kaggle), với mục tiêu chính là minh chứng cho độ chính xác của thuật toán và khả năng xử lý thời gian thực ở mức suy diễn mô hình, chưa phát triển thành sản phẩm thương mại đóng gói hoàn chỉnh.

1.5. Bố cục báo cáo

Báo cáo được trình bày gồm 7 chương với nội dung chi tiết như sau:

Chương 1: Giới thiệu tổng quan. Trình bày lý do chọn đề tài, mục tiêu, đối tượng và phạm vi nghiên cứu của báo cáo môn học.

Chương 2: Cơ sở lý thuyết. Cung cấp nền tảng kiến thức về bài toán nhận dạng hành động, phân tích hạn chế của mạng CNN 2D truyền thống và làm rõ nguyên lý hoạt động,

kiến trúc toán học của mạng nơ-ron tích chập 3 chiều (3D CNN) được sử dụng trong đề tài.

Chương 3: Dữ liệu huấn luyện. Giới thiệu bộ dữ liệu UCF50, lý giải việc lựa chọn tập con dữ liệu và trình bày chi tiết các bước tiền xử lý quan trọng như trích xuất khung hình, chuẩn hóa dữ liệu và phân chia tập huấn luyện - kiểm tra.

Chương 4: Lưu đồ thuật toán. Mô hình hóa quy trình hoạt động của hệ thống thông qua sơ đồ khối, giải thích logic xử lý từ khâu nạp dữ liệu đầu vào đến khâu đánh giá mô hình.

Chương 5: Code chương trình. Mô tả chi tiết môi trường cài đặt, thư viện sử dụng và các khối lệnh quan trọng. Đặc biệt, chương này tập trung phân tích các cải tiến trong mã nguồn của sinh viên so với mã nguồn tham khảo, minh chứng qua kiến trúc mạng và chiến lược lấy mẫu dữ liệu.

Chương 6: Phân tích và đánh giá kết quả. Trình bày kết quả thực nghiệm thông qua biểu đồ Loss/Accuracy, Ma trận nhầm lẫn (Confusion Matrix) và các chỉ số định lượng (Precision, Recall, F1-Score). Chương này cũng bao gồm đánh giá tốc độ xử lý thời gian thực và so sánh hiệu quả cải thiện của mô hình đề xuất so với mô hình cơ sở.

Chương 7: Kết luận và hướng phát triển. Tổng kết các kết quả đạt được, nhìn nhận các hạn chế còn tồn tại và đề xuất các hướng nghiên cứu mở rộng trong tương lai.

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT

2.1. Tổng quan về bài toán nhận dạng hành động (Human Action Recognition)

2.1.1. Định nghĩa bài toán

Nhận dạng hành động con người (Human Action Recognition - HAR) là một bài toán cốt lõi trong lĩnh vực thị giác máy tính, với mục tiêu xây dựng các hệ thống có khả năng tự động phân tích và gán nhãn ngữ nghĩa cho các hành động của con người xuất hiện trong dữ liệu video.

Về mặt toán học, nếu xem một đoạn video V là một tập hợp chuỗi các khung hình liên tiếp $V = \{f_1, f_2, \dots, f_T\}$, nhiệm vụ của bài toán là tìm ra một hàm ánh xạ F sao cho $y = F(V)$, trong đó y là nhãn hành động thuộc tập hợp các lớp hành động đã định nghĩa trước (ví dụ: đi bộ, chạy, vẫy tay).

Khác với bài toán phân loại ảnh tĩnh (Image Classification) chỉ tập trung vào sự hiện diện của đối tượng trong không gian, bài toán nhận dạng hành động đòi hỏi mô hình phải hiểu được sự thay đổi trạng thái của đối tượng đó theo trục thời gian.

2.1.2. Hai thành phần đặc trưng của video

Để giải quyết bài toán HAR hiệu quả, một hệ thống cần phải trích xuất và xử lý đồng thời hai luồng thông tin đặc trưng cơ bản của video:

Đặc trưng không gian (Spatial Features): Đây là thông tin về ngoại hình, bối cảnh, màu sắc và kết cấu của đối tượng trong từng khung hình riêng lẻ. Đặc trưng này giúp trả lời câu hỏi "Đối tượng là ai/cái gì?" và "Bối cảnh ở đâu?". Các mạng 2D CNN truyền thống xử lý rất tốt loại thông tin này.

Đặc trưng thời gian (Temporal Features): Đây là thông tin về sự vận động, hướng di chuyển và tốc độ thay đổi giữa các khung hình liên tiếp. Đặc trưng này giúp trả lời câu hỏi "Đối tượng đang làm gì và diễn biến như thế nào?". Đây là yếu tố quyết định để phân biệt các hành động có bối cảnh giống nhau (ví dụ: đi bộ và chạy bộ).

2.1.3. Các thách thức chính

Xử lý dữ liệu video phức tạp hơn nhiều so với ảnh tĩnh do các yếu tố nhiễu trong môi trường thực tế:

1. Sự đa dạng nội tại (Intra-class variation): Cùng một hành động (ví dụ: "nhảy") có thể được thực hiện bởi nhiều người với vóc dáng, tốc độ và phong cách hoàn toàn khác nhau.

2. Nhiều nền và chuyển động camera (Background clutter & Camera motion): Trong video thực tế, nền thường không cố định và camera có thể bị rung lắc, khiến mô hình khó phân biệt đâu là chuyển động của người, đâu là chuyển động của nền.
3. Độ dài video không cố định: Các video đầu vào có số lượng khung hình (frames) khác nhau, đòi hỏi mô hình phải có cơ chế chuẩn hóa hoặc xử lý chuỗi linh hoạt.

Để vượt qua các thách thức trên, việc sử dụng các mô hình học sâu truyền thống (chỉ dựa trên 2D CNN) là không đủ vì chúng bỏ qua mối liên kết thời gian. Do đó, cần một kiến trúc mạng nơ-ron chuyên biệt có khả năng học đồng thời cả không gian và thời gian, đó là tiền đề cho việc áp dụng Mạng nơ-ron tích chập 3 chiều (3D CNN).

2.2. Mạng nơ-ron tích chập (Convolutional Neural Network – CNN)

2.2.1. Giới thiệu chung

Mạng nơ-ron tích chập (CNN) là một kiến trúc chuyên biệt của mạng nơ-ron nhân tạo, được lấy cảm hứng từ cơ chế hoạt động của vỏ não thị giác sinh học ở động vật. Kể từ khi AlexNet giành chiến thắng vang dội tại cuộc thi ImageNet năm 2012, CNN đã trở thành tiêu chuẩn vàng trong các bài toán xử lý dữ liệu có cấu trúc dạng lưới (grid topology), điển hình nhất là hình ảnh kỹ thuật số (mạng lưới các pixel 2D).

Điểm khác biệt căn bản của CNN so với các mạng nơ-ron truyền thống (Feed-forward Neural Networks) nằm ở khả năng chia sẻ trọng số (weight sharing) và kết nối cục bộ (local connectivity). Điều này cho phép CNN giảm thiểu đáng kể số lượng tham số cần huấn luyện trong khi vẫn duy trì khả năng trích xuất các đặc trưng bất biến với phép tịnh tiến (translation invariance).

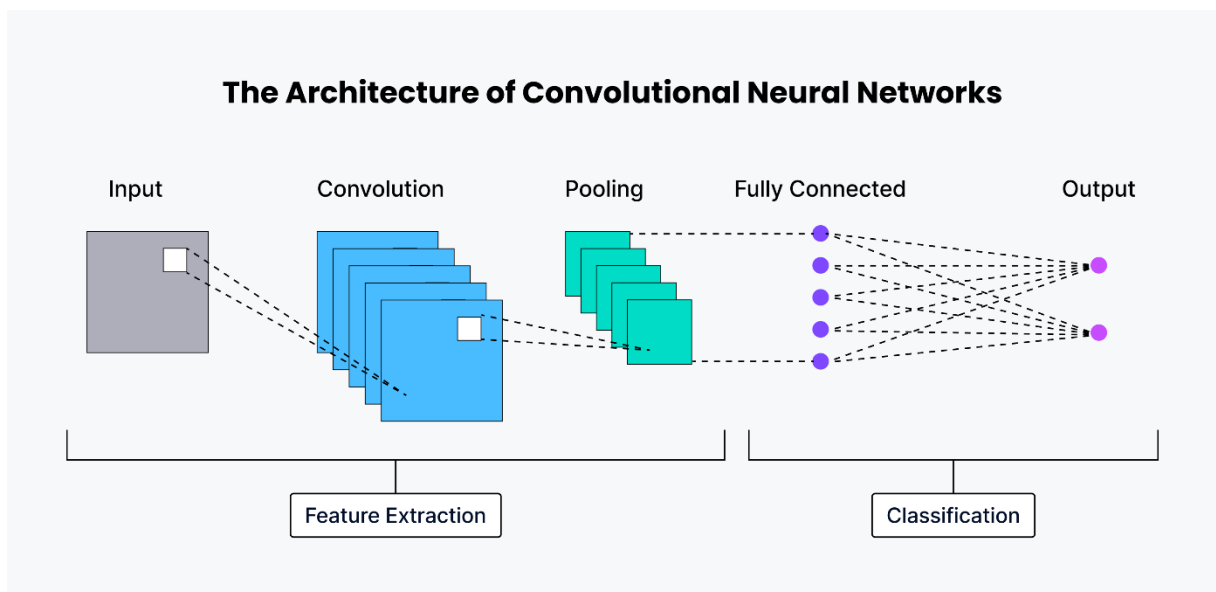
2.2.2. Các thành phần kiến trúc chính

Một mạng CNN điển hình được cấu thành từ chuỗi các lớp xử lý xếp chồng lên nhau, bao gồm ba loại lớp chính:

Lớp tích chập (Convolutional Layer): Đây là thành phần nòng cốt thực hiện chức năng trích xuất đặc trưng. Lớp này sử dụng tập hợp các bộ lọc (filters/kernels) có kích thước nhỏ (ví dụ: 3×3 hoặc 5×5) trượt qua toàn bộ hình ảnh đầu vào. Tại mỗi vị trí, một phép tính tích vô hướng (dot product) giữa các trọng số của kernel và vùng ảnh cục bộ được thực hiện để tạo ra các bản đồ đặc trưng (feature maps). Các bản đồ này đại diện cho các đặc trưng thị giác từ thấp đến cao như cạnh, góc, kết cấu hay hình dạng vật thể.

Lớp gộp (Pooling Layer): Thường được đặt sau các lớp tích chập nhằm mục đích giảm chiều dữ liệu (down-sampling). Phổ biến nhất là Max Pooling (lấy giá trị lớn nhất trong vùng). Lớp này giúp giảm chi phí tính toán, hạn chế hiện tượng quá khớp (overfitting) và giúp mô hình trở nên bất biến với các biến đổi nhỏ về vị trí của đối tượng trong ảnh.

Lớp kết nối đầy đủ (Fully Connected Layer - FC): Nằm ở các tầng cuối cùng của mạng. Sau khi các đặc trưng không gian đã được trích xuất và nén lại bởi các lớp Convolution và Pooling, chúng được làm phẳng (flatten) thành các vector một chiều và đưa vào lớp FC để thực hiện việc phân loại cuối cùng dựa trên xác suất.



Hình 2.1. Cấu trúc tổng quát của mạng nơ-ron tích chập (CNN) gồm các lớp Convolution, Pooling và Fully Connected (Nguồn: Adapted from Reshna S et al., 2025).

2.2.3. Cơ chế hoạt động trên dữ liệu ảnh

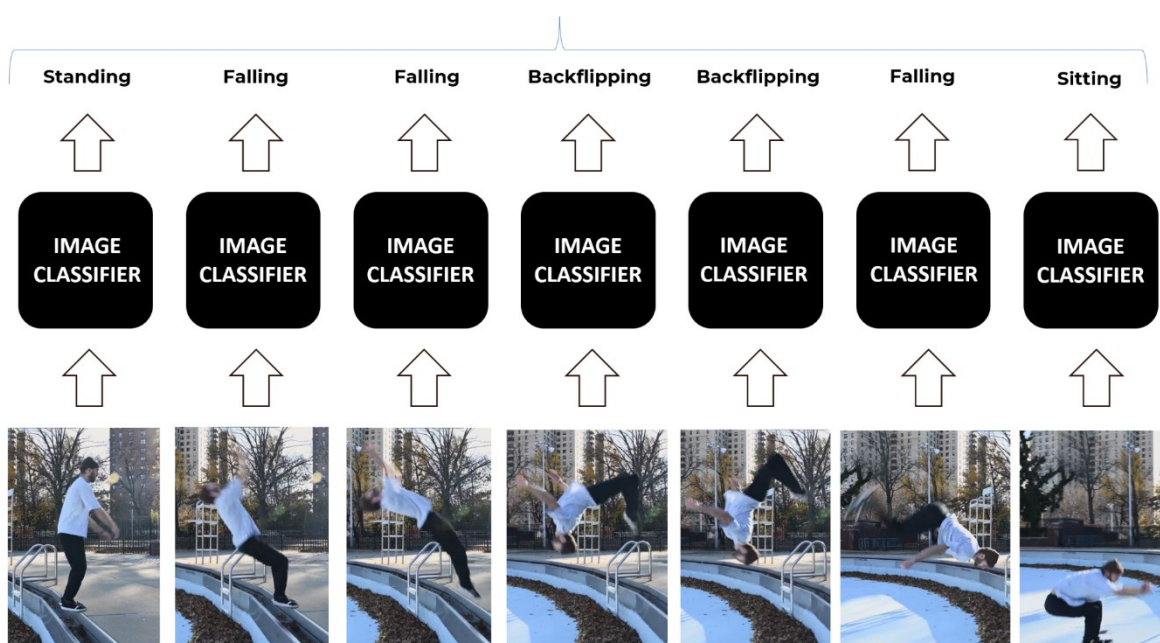
Trong mô hình CNN truyền thống (2D CNN), dữ liệu đầu vào là một ma trận 3 chiều $H \times W \times C$ (Chiều cao $\times$ Chiều rộng $\times$ Số kênh màu). Các phép tính tích chập chỉ diễn ra trên hai chiều không gian (H và W). Điều này có nghĩa là CNN 2D cực kỳ hiệu quả trong việc nhận diện các đặc trưng tĩnh trong một bức ảnh đơn lẻ. Tuy nhiên, kiến trúc này mặc định coi mỗi đầu vào là độc lập, không có bộ nhớ về các trạng thái trước đó, dẫn đến những hạn chế nhất định khi xử lý dữ liệu chuỗi như video.

2.3. Hạn chế của CNN 2D trong xử lý video

Mặc dù các mô hình 2D CNN đã đạt được độ chính xác rất cao trong bài toán phân loại ảnh tĩnh, việc áp dụng trực tiếp kiến trúc này vào dữ liệu video tồn tại một "điểm mù" lớn về mặt cấu trúc: sự thiếu hụt thông tin thời gian.

2.3.1. Vấn đề xử lý độc lập từng khung hình

Về bản chất, video là một chuỗi các khung hình (frames) có mối liên kết chặt chẽ về mặt thời gian. Khi sử dụng 2D CNN cho video, mô hình buộc phải xử lý từng khung hình (frame) như một thực thể độc lập. Hệ thống sẽ đưa ra dự đoán cho từng ảnh riêng lẻ mà không "biết" gì về các khung hình trước hoặc sau nó.



Hình 2.2. Minh họa hạn chế của 2D CNN: Các khung hình được xử lý riêng lẻ, mất đi sự liên kết thời gian. (Nguồn: Adapted from Bleed AI Academy, 2024; <https://bleedaiacademy.com/human-activity-recognition-using-tensorflow-cnn-lstm/>).

Quan sát Hình 2.2, ta thấy một ví dụ điển hình minh họa cho sự hạn chế của mô hình 2D khi nhận dạng hành động nhào lộn (Backflip). Khi một chuỗi khung hình liên tiếp mô tả trọn vẹn hành động được đưa qua bộ phân loại ảnh 2D, hệ thống xử lý từng ảnh một cách độc lập và cho ra các kết quả rất hỗn loạn do thiếu ngữ cảnh tham chiếu. Cụ thể, ở giai đoạn khởi đầu, mô hình chỉ ghi nhận tư thế tĩnh và dự đoán là "Standing" (Đứng). Nghiêm trọng hơn, tại các khung hình giữa khi đối tượng đang thực hiện động tác trên không, sự thiếu hụt thông tin về chuỗi chuyển động khiến mô hình hiểu sai thành hành động "Falling" (Té ngã), và chỉ một số ít khung hình bắt được đúng khoảnh khắc

"Backflipping". Đến khi đối tượng tiếp đất, tư thế này lại bị gán nhãn sai lệch thành "Sitting" (Ngồi). Sự không nhất quán giữa các nhãn dự đoán này cho thấy nếu chỉ dựa vào đặc trưng không gian của từng ảnh đơn lẻ, hệ thống hoàn toàn thất bại trong việc nắm bắt bản chất thực sự của hành động diễn ra theo thời gian.

2.3.2. Mất mát thông tin ngữ nghĩa toàn cục

Nếu sử dụng phương pháp lấy trung bình cộng xác suất (Average of prediction probabilities) như trong hình minh họa, hệ thống rất dễ đưa ra kết luận sai lệch. Trong ví dụ trên, số lượng nhãn "Falling" (Ngã) xuất hiện nhiều ngang ngửa hoặc hơn nhãn "Backflipping". Điều này dẫn đến việc hệ thống có thể kết luận sai hành động của người trong video là "Té ngã" thay vì "Nhào lộn".

Nguyên nhân cốt lõi là 2D CNN chỉ trích xuất được đặc trưng không gian (spatial features) – tức là tư thế, hình dáng tại một thời điểm cố định. Nó thất bại trong việc nắm bắt đặc trưng thời gian (temporal features) – tức là chuỗi vận động liên mạch. Hành động "Nhào lộn" không được định nghĩa bởi một tư thế tĩnh, mà được định nghĩa bởi sự thay đổi tư thế liên tục theo thời gian. Đây chính là lý do bắt buộc phải chuyển sang sử dụng mô hình 3D CNN.

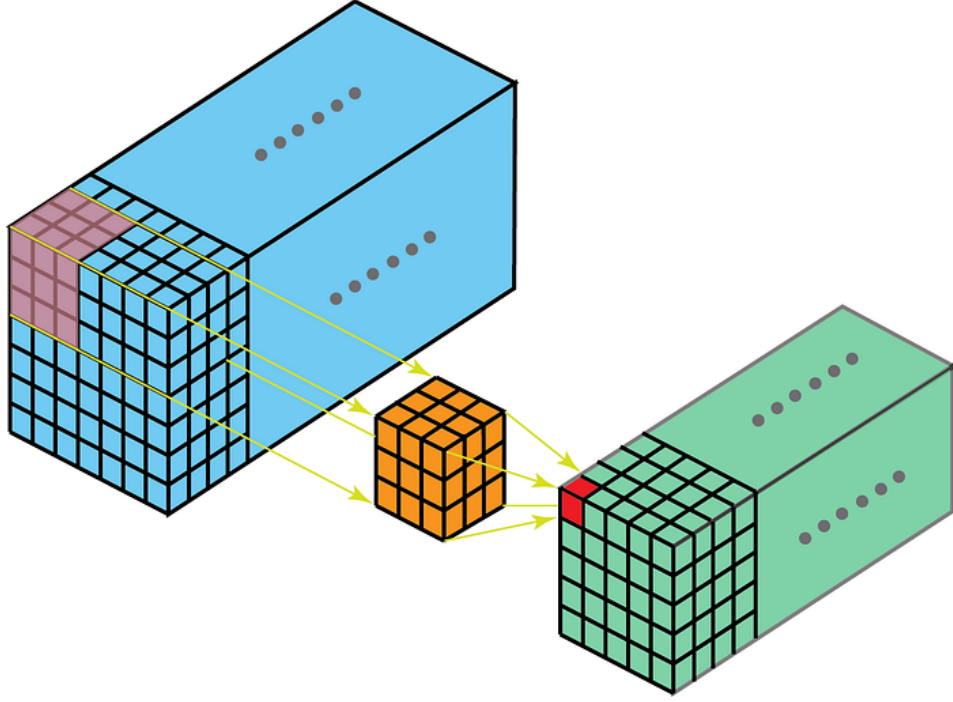
2.4. Mạng nơ-ron tích chập 3 chiều (3D CNN)

Để khắc phục nhược điểm mất mát thông tin thời gian của 2D CNN đã phân tích ở trên, kiến trúc mạng nơ-ron tích chập 3 chiều (3D CNN) được đề xuất và sử dụng làm thuật toán nòng cốt trong báo cáo này.

2.4.1. Khái niệm và nguyên lý hoạt động

Về mặt bản chất, 3D CNN là sự mở rộng tự nhiên của 2D CNN vào miền thời gian. Sự khác biệt cốt lõi nằm ở cấu trúc của bộ lọc (kernel). Thay vì sử dụng các kernel 2 chiều có kích thước $k \times k$ chỉ trượt trên không gian ảnh, 3D CNN sử dụng các kernel khối 3 chiều có kích thước $k \times k \times d$, trong đó d là độ sâu thời gian (temporal depth), tương ứng với số lượng khung hình liên tiếp mà kernel xử lý trong một phép tính.

Trong quá trình hoạt động, các kernel 3D này sẽ thực hiện phép tích chập bằng cách trượt qua khối video đầu vào (Input Volume) theo cả ba phương: chiều rộng, chiều cao và chiều thời gian.



Hình 2.3. Minh họa cơ chế tích chập 3D. Kernel khối (màu vàng) trượt trên khối dữ liệu đầu vào để tạo ra khối đặc trưng đầu ra (output volume). (Nguồn: Adapted from Zhao et al., 2024).

Kết quả của phép tính này là các bản đồ đặc trưng (feature maps) dạng khối 3D. Điều này có ý nghĩa quan trọng: thông tin chuyển động và mối tương quan giữa các khung hình không bị triệt tiêu sau phép tích chập (như trong 2D CNN) mà được bảo toàn và mã hóa trực tiếp vào cấu trúc của dữ liệu đầu ra. Nhờ đó, mạng có khả năng học đồng thời cả đặc trưng về hình dáng (spatial) và đặc trưng về vận động (temporal) trong một kiến trúc thống nhất.

2.4.2. Mô hình toán học

Dựa trên nguyên lý trên, giá trị của một nơ-ron tại vị trí tọa độ (x, y, z) trên bản đồ đặc trưng thứ j ở lớp thứ i , ký hiệu là v_{ij}^{xyz} , được xác định bằng công thức tổng quát sau:

Phương trình 2.1. Giá trị một nơ-ron ở tọa độ (x, y, z) trên bản đồ đặc trưng thứ j ở lớp thứ i

$$v_{ij}^{xyz} = f \left(b_{ij} + \sum_m \sum_{p=0}^{P_i-1} \sum_{q=0}^{Q_i-1} \sum_{r=0}^{R_i-1} w_{ijm}^{pqr} \cdot v_{(i-1)m}^{(x+p)(y+q)(z+r)} \right)$$

Trong công thức này:

$f(\cdot)$: Là hàm kích hoạt phi tuyến. Trong các nghiên cứu hiện đại và trong báo cáo môn học này, chúng tôi ưu tiên sử dụng hàm ReLU ($f(x) = \max(0, x)$) thay cho tanh để tăng tốc độ hội tụ và giảm chi phí tính toán.

b_{ij} : Là giá trị bias (độ lệch) cho bản đồ đặc trưng thứ j .

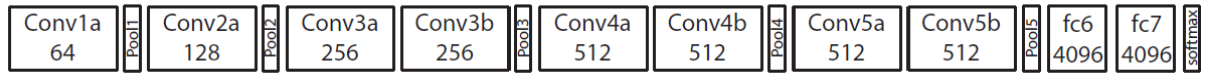
w_{ijm}^{pqr} : Là trọng số (weight) tại vị trí (p, q, r) của kernel thứ j kết nối với bản đồ đặc trưng thứ m ở lớp trước đó ($i - 1$).

P_i, Q_i, R_i : Lần lượt là chiều cao, chiều rộng và độ sâu thời gian của 3D kernel.

Công thức cho thấy mỗi giá trị đầu ra là sự tổng hợp thông tin từ một vùng không gian - thời gian cục bộ, cho phép mạng nắm bắt được các kiểu chuyển động phức tạp.

2.4.3. Kiến trúc C3D tham khảo

Trong khuôn khổ báo cáo, mô hình được xây dựng dựa trên cảm hứng từ kiến trúc C3D (Convolutional 3D) – một trong những mô hình tiên phong chứng minh hiệu quả của 3D CNN. Đặc điểm nổi bật của kiến trúc này là tính đồng nhất: tất cả các lớp tích chập đều sử dụng kernel có kích thước $3 \times 3 \times 3$ với bước trượt (stride) là $1 \times 1 \times 1$.



Hình 2.4. Sơ đồ kiến trúc mạng C3D điển hình (Nguồn: Adapted from Samuel Felipe dos Santos et al., 2019).

Các nghiên cứu thực nghiệm đã chỉ ra rằng kích thước kernel $3 \times 3 \times 3$ là lựa chọn tối ưu nhất để mạng có thể trích xuất đặc trưng không gian – thời gian hiệu quả mà không làm bùng nổ số lượng tham số cần huấn luyện. Mạng thường bao gồm chuỗi các khối Conv3D nối tiếp với các lớp Pooling3D (thường là Max Pooling) để giảm chiều dữ liệu, và kết thúc bằng các lớp kết nối đầy đủ (Fully Connected) để phân loại hành động.

2.4.4. So sánh tổng hợp giữa 2D CNN và 3D CNN

Để làm rõ tính ưu việt của phương pháp được lựa chọn, chúng ta có thể so sánh hai kiến trúc dựa trên các tiêu chí kỹ thuật chính. Trong khi 2D CNN coi video là tập hợp các ảnh rời rạc và đánh mất thông tin chiều sâu thời gian ngay sau lớp tích chập đầu tiên, thì 3D CNN duy trì cấu trúc khối dữ liệu xuyên suốt quá trình trích xuất đặc trưng. Điều này cho phép 3D CNN mô hình hóa được thông tin chuyển động (motion modeling) trực tiếp từ dữ liệu pixel thô mà không cần các bước tiền xử lý phức tạp như tính toán luồng quang học (Optical Flow). Tuy nhiên, sự đánh đổi nằm ở chi phí tính toán: 3D CNN yêu

cầu tài nguyên phần cứng lớn hơn và lượng dữ liệu huấn luyện nhiều hơn để tránh hiện tượng quá khớp (overfitting).

2.5. Các thành phần hỗ trợ trong huấn luyện mô hình

Để quá trình huấn luyện mạng 3D CNN đạt hiệu quả và hội tụ tốt, ngoài kiến trúc chính, cần xác định rõ các thành phần hỗ trợ sau:

2.5.1. Hàm kích hoạt đầu ra (Output Activation)

Bài toán trong báo cáo môn học được xác định là phân loại đa lớp (Multi-class Classification), trong đó mỗi video đầu vào được gán nhãn thuộc về một trong C lớp hành động (ví dụ: Basketball, HorseRiding, v.v.). Do đó, lớp cuối cùng của mạng sử dụng hàm kích hoạt Softmax để chuyển đổi các giá trị logits thành phân bố xác suất.

Phương trình 2.2. Hàm kích hoạt softmax

$$P(y = c|x) = \frac{e^{z_c}}{\sum_{j=1}^C e^{z_j}}$$

Trong đó: z_c là giá trị đầu ra của nơ-ron thứ c trước hàm kích hoạt, và C là tổng số lớp hành động.

2.5.2. Hàm mất mát (Loss Function)

Dựa trên cấu trúc nhãn dữ liệu được mã hóa dưới dạng số nguyên (Integer Encoding) thay vì one-hot vector (nhằm tiết kiệm bộ nhớ khi số lớp lớn), báo cáo sử dụng hàm mất mát Sparse Categorical Cross-Entropy. Hàm này đo lường độ sai lệch giữa phân bố xác suất dự đoán và nhãn thực tế.

Phương trình 2.3. Hàm mất mát Sparse Categorical Cross-Entropy

$$L = -\frac{1}{N} \sum_{i=1}^N y_{i,k} \log(\hat{y}_{i,k})$$

Trong đó:

N : Tổng số mẫu trong batch.

$y_{i,k}$: Nhãn thật của mẫu i đối với lớp k ($y = 1$ khi mẫu thuộc lớp k và $y = 0$ nếu ngược lại).

$\hat{y}_{i,k}$: Xác suất mô hình dự đoán mẫu i thuộc về lớp đúng k .

Mục tiêu của quá trình huấn luyện là cực tiểu hóa giá trị L này.

2.5.3. Thuật toán tối ưu hóa (Optimizer)

Để cập nhật trọng số mạng nhằm giảm thiểu hàm mất mát, chúng tôi sử dụng thuật toán Adam (Adaptive Moment Estimation). Đây là sự kết hợp ưu điểm của hai thuật toán Momentum và RMSProp.

Cơ chế: Adam tính toán tốc độ học thích ứng (adaptive learning rates) cho từng tham số riêng biệt dựa trên các mô-men bậc nhất và bậc hai của gradient.

Lý do lựa chọn: Adam hội tụ nhanh hơn và hoạt động ổn định hơn trên các bài toán có dữ liệu thưa hoặc nhiễu lớn như video so với thuật toán Gradient Descent truyền thống (SGD).

2.6. Đánh giá ưu và nhược điểm của mô hình đề xuất

Dựa trên cơ sở lý thuyết về 3D CNN và kiến trúc mạng C3D rút gọn được sử dụng, có thể đánh giá các đặc điểm sau:

Ưu điểm:

Tính toàn vẹn thông tin: Mô hình trích xuất được đồng thời đặc trưng không gian (hình dáng đối tượng) và thời gian (chuyển động) trong một quá trình duy nhất, không cần tách rời các bước xử lý.

Không cần thiết kế đặc trưng thủ công: Mô hình học trực tiếp từ dữ liệu pixel thô (raw pixels), loại bỏ nhu cầu sử dụng các thuật toán hỗ trợ phức tạp như Optical Flow.

Khả năng tổng quát hóa: Kiến trúc C3D đã được chứng minh là có khả năng hoạt động tốt trên nhiều tập dữ liệu hành động khác nhau mà không cần tinh chỉnh quá nhiều.

Nhược điểm:

Chi phí tính toán lớn: Phép tích chập 3D đòi hỏi khối lượng tính toán và bộ nhớ GPU lớn hơn nhiều so với 2D CNN. Điều này giới hạn độ sâu của mạng khi triển khai trên các thiết bị cấu hình trung bình.

Hiện tượng Overfitting: Do số lượng tham số lớn, mô hình 3D CNN rất dễ bị quá khớp nếu tập dữ liệu huấn luyện nhỏ. (Để khắc phục, báo cáo môn học sẽ áp dụng các lớp Dropout và kỹ thuật Data Augmentation).

CHƯƠNG 3. DỮ LIỆU HUẤN LUYỆN

3.1. Giới thiệu chung về dữ liệu huấn luyện

Trong các bài toán học sâu, đặc biệt là bài toán nhận dạng hành động từ video, dữ liệu huấn luyện đóng vai trò then chốt, quyết định trực tiếp đến khả năng học đặc trưng và hiệu năng của mô hình. Khác với dữ liệu ảnh tĩnh, dữ liệu video có cấu trúc phức tạp hơn do chứa đồng thời thông tin không gian và thời gian, đòi hỏi quá trình xử lý và chuẩn hóa dữ liệu phải được thực hiện cẩn thận trước khi đưa vào huấn luyện.

Trong khuôn khổ đề tài này, dữ liệu được sử dụng nhằm xây dựng và đánh giá một hệ thống mẫu nhận dạng hành động từ video sử dụng mạng nơ-ron tích chập 3 chiều (3D CNN). Do mục tiêu chính của đề tài là nghiên cứu nguyên lý hoạt động của mô hình và quy trình huấn luyện, tập dữ liệu được lựa chọn ở quy mô phù hợp, không hướng đến việc huấn luyện mô hình có khả năng tổng quát hóa cao trong môi trường thực tế phức tạp.

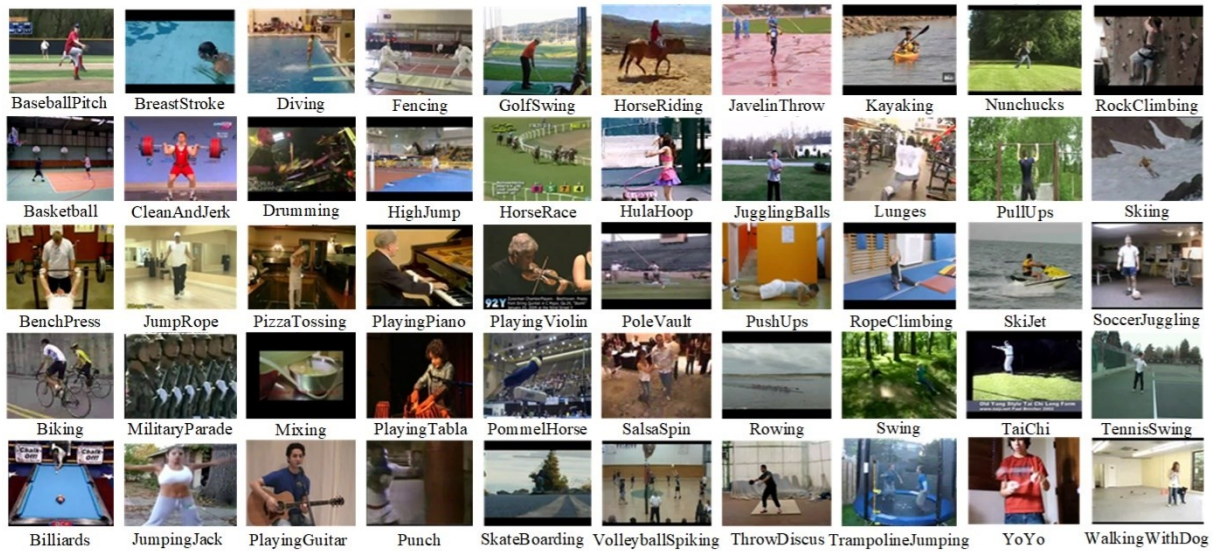
Dữ liệu huấn luyện trong đề tài bao gồm các đoạn video hành động đã được gán nhãn, được sử dụng cho các mục đích:

1. Huấn luyện mô hình học các đặc trưng không gian–thời gian đặc trưng cho từng hành động.
2. Đánh giá khả năng phân loại của mô hình trên các tập dữ liệu chưa từng xuất hiện trong quá trình huấn luyện.
3. Phân tích những ưu điểm và hạn chế của mô hình 3D CNN khi áp dụng cho một hệ thống mẫu có số lượng lớp hành động giới hạn.

Việc lựa chọn và xử lý dữ liệu theo hướng này giúp đề tài tập trung vào tính đúng đắn về mặt phương pháp, đảm bảo mô hình được xây dựng, huấn luyện và đánh giá theo quy trình chuẩn của học sâu, đồng thời tạo nền tảng cho việc mở rộng và cải tiến trong các nghiên cứu tiếp theo.

3.2. Bộ dữ liệu UCF50

Bộ dữ liệu UCF50 là một tập dữ liệu video hành động phổ biến, thường được sử dụng trong các nghiên cứu và bài toán thử nghiệm về nhận dạng hành động người từ video. Dữ liệu bao gồm nhiều đoạn video ghi lại các hành động khác nhau của con người trong các bối cảnh đa dạng, được thu thập từ nhiều nguồn thực tế như internet và video do người dùng tự quay (user-generated videos).



Hình 3.1. Một số mẫu khung hình video trong bộ dữ liệu UCF50 với các hành động khác nhau. (Nguồn: Adapted from Vrskova et al., 2023).

Các video trong UCF50 phản ánh sự đa dạng về:

- Bối cảnh quay (trong nhà, ngoài trời).
- Góc nhìn và khoảng cách camera.
- Điều kiện ánh sáng và nền cảnh.
- Cách thức và tốc độ thực hiện hành động.

Nhờ tính đa dạng này, UCF50 thường được xem là một bộ dữ liệu phù hợp để đánh giá khả năng học đặc trưng không gian–thời gian của các mô hình học sâu xử lý video.

Trong đề tài này, UCF50 được sử dụng như nguồn dữ liệu tham khảo để xây dựng hệ thống mẫu. Thay vì khai thác toàn bộ các lớp hành động trong bộ dữ liệu, đề tài chỉ lựa chọn một tập con các lớp hành động tiêu biểu nhằm giảm độ phức tạp, phù hợp với mục tiêu nghiên cứu nguyên lý hoạt động của mạng 3D CNN và giới hạn tài nguyên tính toán.

Việc sử dụng UCF50 trong phạm vi này giúp đảm bảo:

- Nguồn dữ liệu có xuất xứ rõ ràng và được cộng đồng nghiên cứu sử dụng rộng rãi.
- Dữ liệu đủ đa dạng để kiểm tra khả năng học của mô hình.
- Quy mô dữ liệu phù hợp cho việc huấn luyện và đánh giá một hệ thống mẫu trong khuôn khổ học phần.

3.3. Lựa chọn tập con dữ liệu cho hệ thống mẫu

Mặc dù bộ dữ liệu UCF50 bao gồm nhiều lớp hành động khác nhau, trong khuôn khổ đề tài này, chỉ một tập con gồm 4 lớp hành động được lựa chọn để huấn luyện và đánh giá mô hình. Việc giới hạn số lượng lớp không nhằm mục tiêu tối đa hóa độ chính xác, mà hướng đến xây dựng một hệ thống mẫu phục vụ nghiên cứu và minh họa nguyên lý hoạt động của mạng nơ-ron tích chập 3 chiều (3D CNN).

Các lớp hành động được lựa chọn dựa trên các tiêu chí sau:

- Có đặc trưng chuyển động rõ ràng, giúp mô hình học được mối quan hệ không gian–thời gian.
- Các hành động có sự khác biệt tương đối về mặt động học, hạn chế sự chồng lấn đặc trưng giữa các lớp.
- Phù hợp với quy mô dữ liệu và tài nguyên tính toán trong phạm vi học phần.

Việc sử dụng tập con dữ liệu mang lại một số lợi ích chính:

- Giảm độ phức tạp của bài toán phân loại đa lớp.
- Rút ngắn thời gian huấn luyện và thử nghiệm mô hình.
- Tạo điều kiện thuận lợi cho việc phân tích, đánh giá và so sánh kết quả huấn luyện.

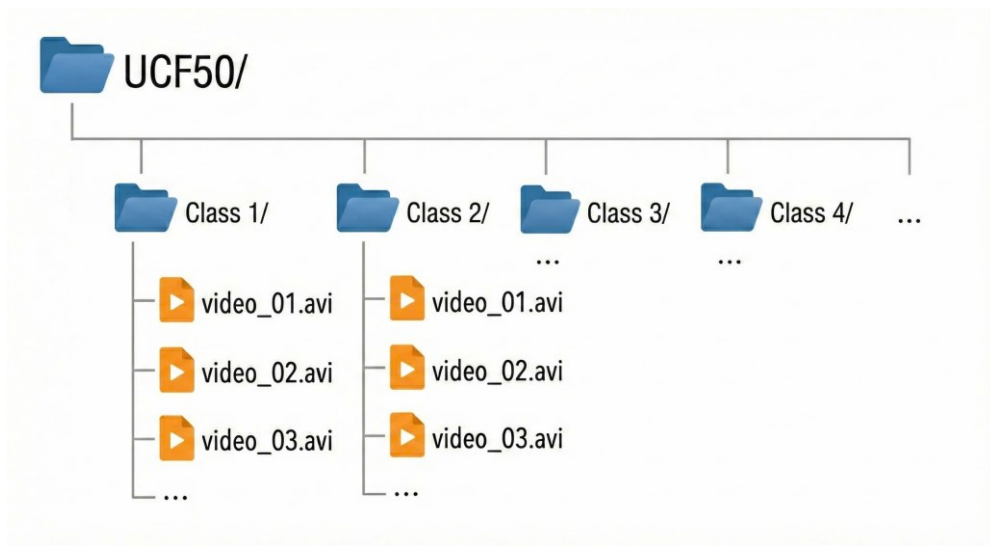
Tuy nhiên, cách tiếp cận này cũng tồn tại những hạn chế nhất định. Do số lượng lớp hành động bị giới hạn, mô hình chưa thể phản ánh đầy đủ sự đa dạng của các hành động trong thực tế, và khả năng tổng quát hóa của hệ thống còn hạn chế. Dù vậy, trong bối cảnh đề tài tập trung vào xây dựng và đánh giá một hệ thống mẫu, việc lựa chọn tập con dữ liệu được xem là phù hợp và hợp lý.

Thông qua cách tiếp cận này, đề tài hướng đến việc làm rõ quy trình xử lý dữ liệu, huấn luyện và đánh giá mô hình 3D CNN, tạo nền tảng cho các nghiên cứu mở rộng với số lượng lớp hành động lớn hơn trong tương lai.

3.4. Cấu trúc và định dạng dữ liệu

Dữ liệu huấn luyện trong đề tài được lưu trữ dưới dạng các tệp video số, mỗi video ghi lại một hành động cụ thể của con người. Các video được tổ chức theo cấu trúc thư mục phân lớp, trong đó mỗi thư mục tương ứng với một lớp hành động. Cách tổ chức này giúp thuận tiện cho việc quản lý dữ liệu, gán nhãn và truy xuất video trong quá trình huấn luyện mô hình.

Cấu trúc tổng quát của tập dữ liệu được tổ chức như sau:



Hình 3.2. Minh họa cấu trúc dữ liệu video và quá trình trích xuất các khung hình từ video đầu vào (Minh họa và tổng hợp bởi tác giả).

Mỗi tệp video bao gồm một chuỗi các khung hình liên tiếp theo thời gian, trong đó số lượng khung hình, độ phân giải và tốc độ khung hình (frame rate) có thể khác nhau giữa các video. Sự không đồng nhất này phản ánh đặc điểm thực tế của dữ liệu video thu thập từ nhiều nguồn khác nhau, nhưng đồng thời cũng đặt ra yêu cầu cần chuẩn hóa dữ liệu trước khi đưa vào mô hình học sâu.

Về mặt định dạng, các video được lưu trữ dưới các chuẩn phổ biến như .avi hoặc .mp4, có thể dễ dàng đọc và xử lý bằng các thư viện xử lý video thông dụng. Trong quá trình huấn luyện, các video không được đưa trực tiếp vào mô hình, mà được chuyển đổi thành các chuỗi khung hình để phục vụ cho việc trích xuất đặc trưng không gian–thời gian.

Việc mô tả rõ cấu trúc và định dạng dữ liệu giúp đảm bảo quá trình tiền xử lý, phân chia tập dữ liệu và huấn luyện mô hình được thực hiện một cách nhất quán, đồng thời tạo nền tảng cho các bước chuẩn hóa và trích xuất dữ liệu đầu vào trong các mục tiếp theo.

3.5. Đặc trưng đầu vào và đầu ra của mô hình

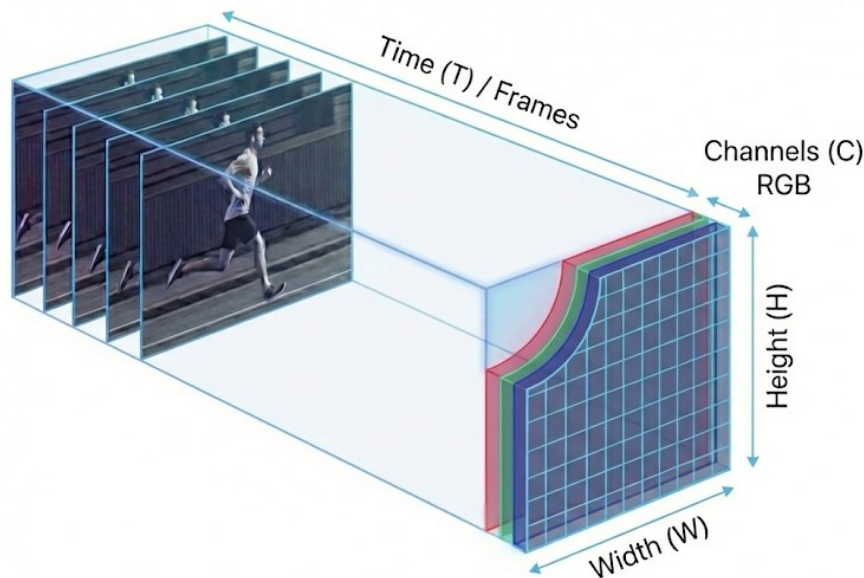
Trong bài toán nhận dạng hành động từ video, việc xác định rõ đặc trưng đầu vào (input features) và đầu ra (output) của mô hình là bước quan trọng nhằm đảm bảo sự phù hợp giữa dữ liệu và kiến trúc mạng nơ-ron tích chập 3 chiều (3D CNN).

Đặc trưng đầu vào

Đầu vào của mô hình là một đoạn video ngắn được biểu diễn dưới dạng chuỗi các khung hình liên tiếp theo thời gian. Để đảm bảo tính nhất quán trong quá trình huấn luyện, mỗi video được chuyển đổi thành một tập khung hình có số lượng cố định. Mỗi khung hình sau khi trích xuất được chuẩn hóa về kích thước và định dạng nhằm phù hợp với yêu cầu của mô hình.

Cụ thể, dữ liệu đầu vào của mô hình có dạng một tensor nhiều chiều, bao gồm:

- Chiều thời gian: biểu diễn số lượng khung hình liên tiếp được sử dụng cho mỗi video.
- Chiều không gian: chiều cao và chiều rộng của khung hình sau khi chuẩn hóa.
- Chiều kênh màu: biểu diễn thông tin màu sắc của khung hình.



Hình 3.3. Cấu trúc dữ liệu đầu vào của mô hình (Minh họa và tổng hợp bởi tác giả).

Việc sử dụng chuỗi khung hình liên tiếp cho phép mô hình 3D CNN học được đồng thời các đặc trưng không gian (hình dạng, tư thế) và thời gian (chuyển động, trình tự hành động) từ dữ liệu video đầu vào.

Đầu ra của mô hình

Đầu ra của mô hình là một vector xác suất biểu diễn khả năng video đầu vào thuộc về từng lớp hành động đã được định nghĩa trước. Số chiều của vector đầu ra tương ứng với số lượng lớp hành động trong hệ thống mẫu.

Trong bài toán này, đầu ra được xây dựng theo dạng phân loại đa lớp (multi-class classification), trong đó:

- Mỗi phần tử của vector đầu ra thể hiện xác suất tương ứng với một lớp hành động.
- Lớp hành động có xác suất cao nhất được xem là kết quả dự đoán của mô hình.

Cách biểu diễn đầu vào và đầu ra theo cấu trúc này giúp mô hình dễ dàng tích hợp với các hàm mất mát và thuật toán tối ưu trong quá trình huấn luyện, đồng thời tạo điều kiện thuận lợi cho việc đánh giá và phân tích kết quả ở các chương tiếp theo.

3.6. Các bước tiền xử lý dữ liệu

Do dữ liệu video có cấu trúc phức tạp và không đồng nhất về độ dài, độ phân giải cũng như tốc độ khung hình, các bước tiền xử lý đóng vai trò quan trọng nhằm chuẩn hóa dữ liệu đầu vào trước khi đưa vào huấn luyện mô hình 3D CNN. Trong đề tài này, quy trình tiền xử lý dữ liệu được thực hiện theo các bước chính như sau.

Trích xuất khung hình từ video

Mỗi video trong tập dữ liệu được xử lý để trích xuất các khung hình riêng lẻ theo thứ tự thời gian. Việc trích xuất khung hình giúp chuyển đổi dữ liệu video liên tục thành dạng dữ liệu rời rạc, thuận tiện cho quá trình xử lý và huấn luyện mô hình học sâu.

Chuẩn hóa số lượng khung hình

Do các video có độ dài khác nhau, để đảm bảo dữ liệu đầu vào có cấu trúc thống nhất, mỗi video được biểu diễn bằng một chuỗi liên tiếp gồm số lượng khung hình cố định. Chuỗi khung hình này được trích xuất ngẫu nhiên nhưng liên tục theo trục thời gian. Đối với các video có số lượng khung hình nhỏ hơn ngưỡng yêu cầu, video đó sẽ bị loại khỏi tập dữ liệu nhằm đảm bảo tính nhất quán của dữ liệu huấn luyện.

Chuẩn hóa kích thước khung hình

Các khung hình sau khi trích xuất được thay đổi kích thước (resize) về cùng một độ phân giải cố định. Bước này nhằm đảm bảo tính nhất quán của dữ liệu đầu vào và phù hợp với kiến trúc của mạng 3D CNN, đồng thời giúp giảm chi phí tính toán trong quá trình huấn luyện.

Chuẩn hóa giá trị điểm ảnh

Giá trị điểm ảnh của các khung hình được chuẩn hóa về cùng một miền giá trị, thường bằng cách chia cho giá trị tối đa của điểm ảnh. Việc chuẩn hóa này giúp quá trình huấn luyện trở nên ổn định hơn và tăng tốc độ hội tụ của mô hình.

Gán nhãn dữ liệu

Mỗi chuỗi khung hình sau khi tiền xử lý được gán một nhãn tương ứng với lớp hành động của video gốc. Các nhãn này được mã hóa phù hợp với bài toán phân loại đa lớp để phục vụ cho quá trình huấn luyện và đánh giá mô hình.

Thông qua các bước tiền xử lý trên, dữ liệu video ban đầu được chuyển đổi thành dạng dữ liệu có cấu trúc đồng nhất, phù hợp với đầu vào của mạng 3D CNN. Quy trình này không chỉ giúp cải thiện hiệu quả huấn luyện mà còn đảm bảo tính nhất quán và khả năng tái lập của các thí nghiệm trong đề tài.

3.7. Phân chia tập dữ liệu huấn luyện và kiểm tra

Sau khi hoàn tất quá trình tiền xử lý và trích xuất đặc trưng từ dữ liệu video, tập dữ liệu thu được được tiến hành phân chia thành tập huấn luyện (training set) và tập kiểm tra (test set). Việc phân chia này nhằm đánh giá khả năng tổng quát hóa của mô hình trên các dữ liệu chưa từng xuất hiện trong quá trình huấn luyện.

Trong đề tài này, quá trình phân chia dữ liệu được thực hiện ngẫu nhiên trên toàn bộ tập dữ liệu đã tiền xử lý, đảm bảo rằng các mẫu dữ liệu thuộc mỗi lớp hành động đều được phân bố vào cả hai tập. Việc chia dữ liệu được thực hiện trực tiếp trên các cặp dữ liệu đầu vào và nhãn tương ứng, giúp giữ nguyên mối liên hệ giữa chuỗi khung hình và lớp hành động của chúng.

Tập huấn luyện được sử dụng để:

- Huấn luyện mô hình 3D CNN và cập nhật các trọng số thông qua thuật toán tối ưu.
- Giúp mô hình học các đặc trưng không gian–thời gian đặc trưng cho từng lớp hành động.

Tập kiểm tra được sử dụng để:

- Đánh giá hiệu năng của mô hình sau quá trình huấn luyện.
- Phân tích khả năng phân loại của mô hình trên dữ liệu chưa từng được thấy trước đó.

Trong khuôn khổ đề tài, việc phân chia dữ liệu thành hai tập chính (huấn luyện và kiểm tra) là phù hợp với mục tiêu xây dựng hệ thống mẫu và đánh giá bước đầu hiệu quả của mô hình. Việc sử dụng thêm tập xác thực (validation set) hoặc các chiến lược đánh giá nâng cao hơn có thể được xem xét trong các nghiên cứu mở rộng ở tương lai.

3.8. Nhận xét và đánh giá tập dữ liệu

Tập dữ liệu được sử dụng trong đề tài có một số ưu điểm và hạn chế nhất định, ảnh hưởng trực tiếp đến quá trình huấn luyện và đánh giá mô hình 3D CNN.

Ưu điểm của tập dữ liệu

Trước hết, dữ liệu video được lựa chọn từ bộ UCF50 có nguồn gốc rõ ràng và đã được sử dụng rộng rãi trong các nghiên cứu về nhận dạng hành động. Các video chứa nhiều bối cảnh, góc quay và cách thức thực hiện hành động khác nhau, giúp mô hình có cơ hội tiếp cận với sự đa dạng nhất định trong dữ liệu đầu vào.

Bên cạnh đó, việc lựa chọn các lớp hành động có đặc trưng chuyển động tương đối rõ ràng giúp mô hình 3D CNN dễ dàng học được các đặc trưng không gian–thời gian cơ bản, phù hợp với mục tiêu xây dựng và đánh giá một hệ thống mẫu.

Hạn chế của tập dữ liệu

Tuy nhiên, tập dữ liệu trong đề tài cũng tồn tại một số hạn chế. Do chỉ sử dụng một tập con gồm số lượng lớp hành động giới hạn, dữ liệu chưa thể phản ánh đầy đủ sự đa dạng của các hành động trong thực tế. Ngoài ra, số lượng video giữa các lớp có thể chưa hoàn toàn cân bằng, dẫn đến khả năng mô hình học lệch về một số lớp nhất định.

Bên cạnh đó, việc loại bỏ các video có độ dài ngắn hơn ngưỡng yêu cầu trong quá trình tiền xử lý có thể làm giảm thêm số lượng mẫu huấn luyện, ảnh hưởng nhất định đến khả năng tổng quát hóa của mô hình.

3.9. Kết luận chương

Trong chương này, dữ liệu huấn luyện phục vụ cho hệ thống mẫu nhận dạng hành động từ video đã được trình bày một cách chi tiết và có hệ thống. Cụ thể, chương đã giới thiệu nguồn dữ liệu sử dụng, lý do lựa chọn tập con dữ liệu với số lượng lớp hành động giới hạn, cũng như mô tả cấu trúc, định dạng và đặc trưng đầu vào – đầu ra của mô hình.

Bên cạnh đó, các bước tiền xử lý dữ liệu quan trọng như trích xuất khung hình, chuẩn hóa kích thước, chuẩn hóa giá trị điểm ảnh và phân chia tập dữ liệu huấn luyện – kiểm tra cũng đã được trình bày rõ ràng, đảm bảo dữ liệu đầu vào phù hợp với kiến trúc mạng nơ-ron tích chập 3 chiều (3D CNN). Những ưu điểm và hạn chế của tập dữ liệu được phân tích nhằm làm rõ phạm vi và ý nghĩa của các kết quả đạt được trong đề tài.

Thông qua chương này, có thể thấy rằng dữ liệu huấn luyện đã được chuẩn bị theo một quy trình nhất quán và phù hợp với mục tiêu xây dựng hệ thống mẫu, tạo nền tảng

cần thiết cho quá trình huấn luyện và đánh giá mô hình. Trên cơ sở đó, chương tiếp theo sẽ trình bày lưu đồ thuật toán huấn luyện mô hình, nhằm minh họa trực quan các bước xử lý và huấn luyện trong toàn bộ hệ thống.

CHƯƠNG 4. LƯU ĐỒ THUẬT TOÁN

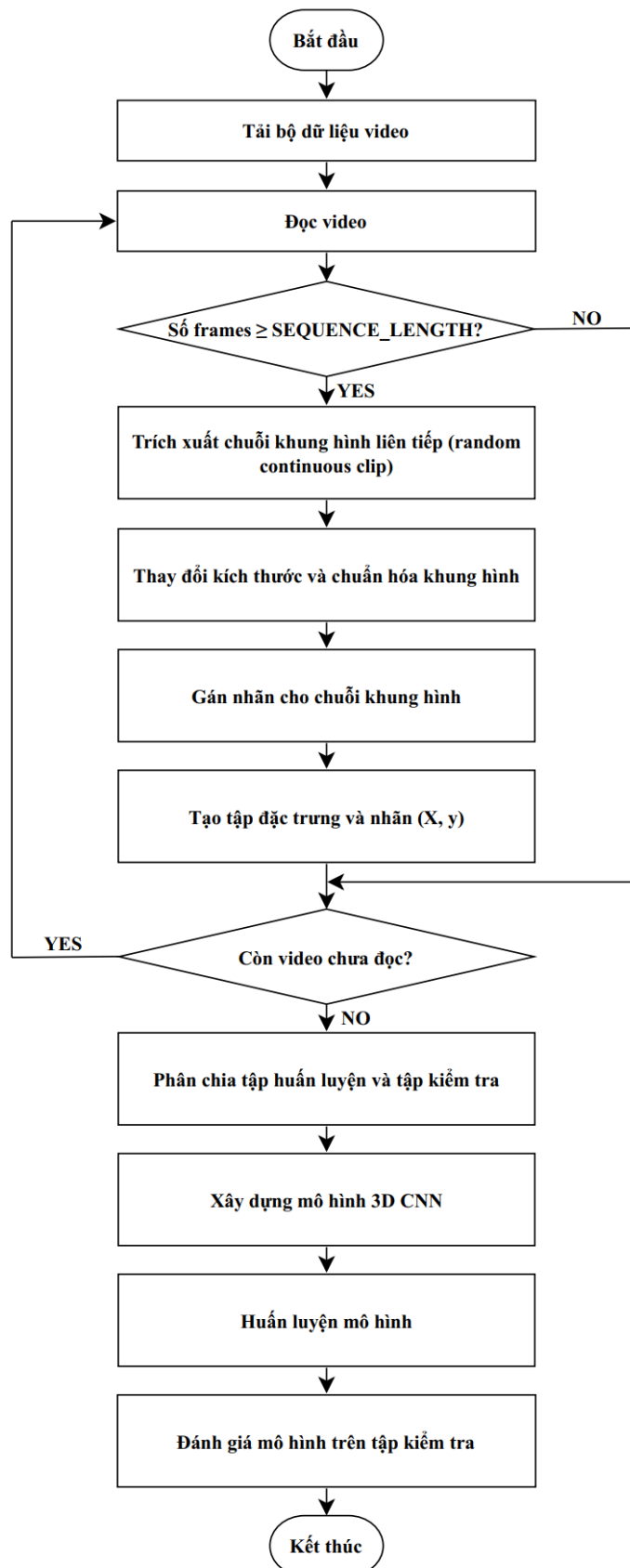
4.1. Tổng quan thuật toán huấn luyện

Quy trình huấn luyện mô hình trong đề tài được xây dựng dựa trên chuỗi các bước xử lý dữ liệu video, tiền xử lý, huấn luyện và đánh giá mô hình 3D CNN. Toàn bộ quy trình được thiết kế theo hướng tuần tự và khép kín, đảm bảo dữ liệu đầu vào được chuẩn hóa trước khi đưa vào mô hình, đồng thời kết quả huấn luyện có thể được đánh giá một cách rõ ràng.

Lưu đồ thuật toán được xây dựng nhằm:

- Minh họa trực quan toàn bộ pipeline huấn luyện.
- Làm rõ mối liên hệ giữa các bước xử lý dữ liệu và huấn luyện mô hình.
- Giúp việc phân tích và triển khai chương trình trở nên dễ hiểu và nhất quán.

4.2. Lưu đồ thuật toán huấn luyện mô hình



Hình 4.1. Lưu đồ thuật toán sử dụng mô hình 3D CNN cho bài toán HAR (Minh họa bởi tác giả)

4.3. Giải thích chi tiết các bước trong lưu đồ

Lưu đồ thuật toán mô tả toàn bộ quy trình huấn luyện mô hình nhận dạng hành động từ video sử dụng mạng nơ-ron tích chập 3 chiều (3D CNN). Quy trình được thực hiện theo các bước tuần tự, phản ánh đúng logic xử lý dữ liệu và huấn luyện trong chương trình.

Bước 1: Bắt đầu và tải bộ dữ liệu video

Thuật toán bắt đầu bằng việc khởi tạo chương trình và tải bộ dữ liệu video từ thư mục lưu trữ. Các video được tổ chức theo từng lớp hành động, tạo điều kiện thuận lợi cho việc gán nhãn trong các bước tiếp theo.

Bước 2: Đọc video

Tại bước này, chương trình lần lượt đọc từng tệp video trong bộ dữ liệu. Việc xử lý dữ liệu được thực hiện theo từng video riêng lẻ nhằm đảm bảo mỗi mẫu dữ liệu đầu vào tương ứng với một hành động cụ thể.

Bước 3: Kiểm tra số lượng khung hình

Sau khi đọc video, thuật toán kiểm tra điều kiện số lượng khung hình của video có lớn hơn hoặc bằng giá trị `SEQUENCE_LENGTH` hay không. Đây là điều kiện cần thiết để đảm bảo mỗi video có đủ số khung hình cho việc trích xuất chuỗi đầu vào của mô hình 3D CNN.

Nếu điều kiện không thỏa mãn (NO): video sẽ bị loại bỏ và chương trình chuyển sang đọc video tiếp theo.

Nếu điều kiện thỏa mãn (YES): video sẽ được đưa vào bước xử lý tiếp theo.

Bước 4: Trích xuất chuỗi khung hình liên tiếp

Đối với các video thỏa mãn điều kiện về số lượng khung hình, chương trình tiến hành trích xuất một chuỗi khung hình liên tiếp theo thời gian, được chọn ngẫu nhiên trong video (random continuous clip). Chuỗi khung hình này đại diện cho dữ liệu đầu vào của mô hình và giúp mô hình học được đặc trưng chuyển động theo thời gian.

Bước 5: Tiền xử lý khung hình

Các khung hình trong chuỗi trích xuất được tiền xử lý thông qua hai thao tác chính: Thay đổi kích thước khung hình về độ phân giải cố định.

Chuẩn hóa giá trị điểm ảnh nhằm đưa dữ liệu về cùng miền giá trị.

Bước này giúp đảm bảo dữ liệu đầu vào có cấu trúc đồng nhất và phù hợp với kiến trúc của mạng 3D CNN.

Bước 6: Gán nhãn cho chuỗi khung hình

Sau khi tiền xử lý, mỗi chuỗi khung hình được gán một nhãn tương ứng với lớp hành động của video ban đầu. Nhãn này được sử dụng trong quá trình huấn luyện để mô hình học mối quan hệ giữa dữ liệu đầu vào và lớp hành động.

Bước 7: Tạo tập đặc trưng và nhãn (X, y)

Các chuỗi khung hình đã được xử lý cùng với nhãn tương ứng được lưu trữ vào các tập dữ liệu đặc trưng (X) và nhãn (y). Quá trình này được lặp lại cho đến khi tất cả các video trong bộ dữ liệu đã được xử lý.

Bước 8: Kiểm tra còn video chưa đọc

Thuật toán kiểm tra xem còn video nào trong bộ dữ liệu chưa được xử lý hay không.

Nếu còn video (YES): chương trình quay lại bước đọc video và tiếp tục xử lý.

Nếu không còn video (NO): quá trình xử lý dữ liệu kết thúc và chuyển sang giai đoạn huấn luyện mô hình.

Bước 9: Phân chia tập huấn luyện và tập kiểm tra

Sau khi hoàn tất việc tạo tập dữ liệu (X, y), dữ liệu được phân chia thành tập huấn luyện và tập kiểm tra. Việc phân chia này nhằm phục vụ cho quá trình huấn luyện mô hình và đánh giá hiệu năng trên dữ liệu chưa từng xuất hiện trong quá trình huấn luyện.

Bước 10: Xây dựng mô hình 3D CNN

Mô hình mạng nơ-ron tích chập 3 chiều được xây dựng với các lớp tích chập 3D, các lớp gộp và các lớp kết nối đầy đủ. Kiến trúc mô hình được thiết kế phù hợp với bài toán phân loại đa lớp trong hệ thống mẫu.

Bước 11: Huấn luyện mô hình

Mô hình được huấn luyện trên tập huấn luyện bằng cách tối ưu hàm mất mát thông qua thuật toán tối ưu. Trong quá trình này, các trọng số của mô hình được cập nhật dần nhằm cải thiện khả năng phân loại hành động.

Bước 12: Đánh giá mô hình trên tập kiểm tra

Sau khi hoàn tất huấn luyện, mô hình được đánh giá trên tập kiểm tra. Kết quả đánh giá được sử dụng để phân tích hiệu năng của mô hình và làm cơ sở cho phần phân tích kết quả trong các chương tiếp theo.

Bước 13: Kết thúc

Thuật toán kết thúc sau khi hoàn thành quá trình đánh giá mô hình.

CHƯƠNG 5. CODE CHƯƠNG TRÌNH

5.1. Môi trường cài đặt và thư viện sử dụng

Để đảm bảo hiệu năng tính toán cho mô hình học sâu 3D CNN, chương trình được triển khai trên nền tảng Kaggle Notebook. Đây là môi trường đám mây cung cấp sẵn tài nguyên phần cứng mạnh mẽ, đặc biệt là GPU NVIDIA Tesla P100 giúp tăng tốc quá trình huấn luyện mô hình xử lý video vốn rất tốn kém tài nguyên nếu chạy trên CPU thông thường.

Bước đầu tiên trong quy trình xây dựng chương trình là thiết lập các công cụ cần thiết. Thay vì nạp toàn bộ thư viện một cách dư thừa, chương trình chỉ trích xuất các module cụ thể phục vụ trực tiếp cho việc xử lý video và xây dựng mạng nơ-ron. Các thư viện được khai báo và nhập vào hệ thống như đoạn mã dưới đây:

```
# Cài đặt thư viện xử lý video (nếu chưa có)
!pip install moviepy

# Nhập các thư viện cốt lõi
import os
import cv2
import random
import numpy as np
import tensorflow as tf

# Thư viện trực quan hóa
import matplotlib.pyplot as plt
%matplotlib inline

# Thư viện Machine Learning để chia tập dữ liệu và đánh giá
from sklearn.model_selection import train_test_split
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay,
classification_report

# Các lớp xây dựng mô hình Deep Learning (Keras)
from tensorflow.keras.models import Model
from tensorflow.keras.layers import (
    Input,
    Conv3D,
```

```

        MaxPooling3D,
        GlobalAveragePooling3D,
        Dense,
        Dropout,
    )
    from tensorflow.keras.callbacks import EarlyStopping, ModelCheckpoint,
    ReduceLROnPlateau
    from tensorflow.keras.utils import to_categorical, plot_model

```

Trong khối lệnh trên, vai trò của từng nhóm thư viện được xác định rõ ràng để phục vụ luồng xử lý dữ liệu. Thư viện OpenCV (cv2) đóng vai trò nòng cốt trong việc đọc video và xử lý ảnh số từng khung hình, trong khi Numpy chịu trách nhiệm thao tác với các mảng dữ liệu nhiều chiều. Quan trọng nhất là nhóm thư viện TensorFlow và Keras cung cấp các lớp chuyên biệt để xây dựng kiến trúc mạng 3D CNN như Conv3D cho tích chập không gian - thời gian, MaxPooling3D để giảm chiều dữ liệu, và GlobalAveragePooling3D để tối ưu hóa đầu ra thay vì dùng lớp Flatten truyền thống. Bên cạnh đó, các công cụ từ Sklearn hỗ trợ việc phân chia dữ liệu kiểm thử và đánh giá chi tiết hiệu quả mô hình thông qua các chỉ số như Confusion Matrix.

Sau khi hoàn tất việc nạp thư viện, chương trình tiến hành thiết lập các tham số cấu hình toàn cục. Việc định nghĩa các hằng số này ngay từ đầu giúp đảm bảo tính nhất quán cho dữ liệu đầu vào và khả năng tái lập kết quả của mô hình.

```

# Thiết lập seed để đảm bảo tính tái lập (Reproducibility)
SEED = 27
np.random.seed(SEED)
random.seed(SEED)
tf.random.set_seed(SEED)

# Đường dẫn dữ liệu
DATASET_DIR = "/kaggle/input/ucf50/UCF50"

# Kích thước chuẩn hóa ảnh đầu vào
IMAGE_HEIGHT = 64
IMAGE_WIDTH = 64

# Độ dài chuỗi khung hình (số frame lấy từ mỗi video)
SEQUENCE_LENGTH = 20

```

```
# Danh sách các hành động cần nhận diện
```

```
CLASSES_LIST = ["WalkingWithDog", "TaiChi", "Swing", "HorseRace"]
```

Các tham số trên đóng vai trò quyết định đến cấu trúc dữ liệu đưa vào mạng nơ-ron. Cụ thể, kích thước khung hình được chuẩn hóa về 64×64 pixels thông qua hai biến `IMAGE_HEIGHT` và `IMAGE_WIDTH`, đây là mức kích thước tối ưu giúp cân bằng giữa độ chi tiết của đặc trưng hình ảnh và tốc độ xử lý của mạng 3D. Tham số `SEQUENCE_LENGTH = 20` quy định mỗi video đầu vào sẽ được lấy mẫu đúng 20 khung hình liên tiếp, giúp mô hình học được sự thay đổi của hành động theo thời gian một cách đồng bộ. Ngoài ra, việc thiết lập hạt giống ngẫu nhiên (`SEED = 27`) cho cả Numpy, Python Random và TensorFlow là bắt buộc để đảm bảo kết quả huấn luyện ổn định và không bị thay đổi ngẫu nhiên giữa các lần chạy khác nhau. Cuối cùng, danh sách `CLASSES_LIST` xác định phạm vi bài toán với 4 lớp hành động cụ thể cần phân loại là Dắt chó đi dạo, Thái cực quyền, Đu dây và Đua ngựa.

5.2. Nguồn code tham khảo và các cải tiến chính

Trong quá trình nghiên cứu và thực hiện báo cáo môn học, sinh viên đã tham khảo bài hướng dẫn cơ sở mang tên *"Human Activity Recognition using TensorFlow (CNN + LSTM)"* của các tác giả Taha Anwar, Rizwan Naeem và Momin Anjum trên trang Bleed AI. Mã nguồn tham khảo này cung cấp một bộ khung cơ bản để xử lý video và xây dựng mô hình Deep Learning sử dụng kiến trúc lai ghép giữa mạng tích chập (CNN) và mạng bộ nhớ ngắn hạn dài hạn (LSTM).

Tuy nhiên, nhận thấy những hạn chế của mã nguồn tham khảo trong việc tối ưu hóa tài nguyên và khả năng bao quát thông tin video, sinh viên thực hiện đã tiến hành tái cấu trúc lại toàn bộ chương trình. Mức độ thay đổi và cải tiến lên đến khoảng 60%, tập trung vào hai khía cạnh cốt lõi là chiến lược tiền xử lý dữ liệu và kiến trúc mạng nơ-ron.

Thứ nhất, về cải tiến trong Tiền xử lý dữ liệu (Data Preprocessing):

Mã nguồn tham khảo thường sử dụng phương pháp đọc tuần tự đơn giản: luôn lấy các khung hình từ giây đầu tiên của video (frame 0 đến frame N). Phương pháp này có nhược điểm là mô hình sẽ bị "học vẹt" phần mở đầu và bỏ lỡ hoàn toàn hành động chính nếu nó diễn ra ở giữa hoặc cuối video.

Để khắc phục, chương trình của sinh viên đã áp dụng chiến lược Random Continuous Sampling (Lấy mẫu ngẫu nhiên đoạn liên tiếp). Thay vì cố định điểm bắt đầu, thuật toán

sẽ chọn ngẫu nhiên một thời điểm bất kỳ trong video (start_frame) và cắt ra chuỗi frame liên tiếp từ đó. Kỹ thuật này đóng vai trò như một phương pháp Tăng cường dữ liệu (Data Augmentation) tự nhiên, giúp mô hình học được hành động ở nhiều bối cảnh thời gian khác nhau, tránh hiện tượng bias (thiên kiến) vào phần đầu video.

Sự khác biệt này được thể hiện rõ qua đoạn code xử lý cải tiến dưới đây:

```
# CODE CẢI TIẾN: Lấy mẫu ngẫu nhiên (Random Sampling)
def frames_extraction(video_path):
    # ... (Khai báo)
    total_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))

    # Cải tiến: Chọn ngẫu nhiên điểm bắt đầu thay vì luôn lấy từ 0
    max_start = total_frames - SEQUENCE_LENGTH
    start_frame = random.randint(0, max_start)

    # Nhảy đến vị trí ngẫu nhiên đó
    cap.set(cv2.CAP_PROP_POS_FRAMES, start_frame)

    for _ in range(SEQUENCE_LENGTH):
        success, frame = cap.read()
        # ... (xử lý tiếp theo)
```

Thứ hai, về cải tiến trong Kiến trúc mô hình (Model Architecture):

Đây là thay đổi mang tính đột phá nhất. Mã nguồn tham khảo sử dụng kiến trúc ConvLSTM2D kết hợp với Flatten. Nhược điểm của Flatten là tạo ra vector đặc trưng khổng lồ, gây tốn bộ nhớ và dễ bị Overfitting.

Chương trình của sinh viên đã thay thế hoàn toàn bằng kiến trúc 3D CNN (Conv3D) sử dụng Functional API (linh hoạt hơn Sequential truyền thống). Đặc biệt, sinh viên thay thế lớp Flatten bằng lớp GlobalAveragePooling3D. Kỹ thuật này nén toàn bộ bản đồ đặc trưng bằng cách tính trung bình, giúp giảm số lượng tham số hàng chục lần, làm mô hình nhẹ hơn và huấn luyện hiệu quả hơn.

Sự tương phản được minh chứng qua hai đoạn mã dưới đây:

Mô hình tham khảo (ConvLSTM + Flatten - Dạng Sequential):

```
model.add(ConvLSTM2D(32, (3, 3)))
model.add(Flatten()) # Nhược điểm: Quá nhiều tham số
model.add(Dense(len(CLASSES_LIST), activation='softmax'))
```

Mô hình cải tiến của sinh viên (3D CNN + Global Pooling - Dạng Functional):

```
# Code thực tế: Sử dụng Functional API và GlobalAveragePooling3D
def create_3d_cnn_model():
    input_layer = Input(shape=(SEQUENCE_LENGTH, IMAGE_HEIGHT, IMAGE_WIDTH,
3))

    # Sử dụng Conv3D thay vì ConvLSTM
    x = Conv3D(32, (3,3,3), activation='relu', padding='same')(input_layer)
    x = MaxPooling3D((1,2,2))(x) # Giữ nguyên trục thời gian ở lớp đầu

    # ... (Các lớp Conv3D sâu hơn) ...

    # Cải tiến quan trọng: Dùng GlobalAveragePooling3D
    x = GlobalAveragePooling3D()(x)

    output = Dense(NUM_CLASSES, activation='softmax')(x)
    return Model(input_layer, output)
```

Tổng hợp lại, việc chuyển đổi chiến lược lấy mẫu (Random Sampling) và hiện đại hóa kiến trúc (3D CNN Functional + Global Pooling) đã tạo nên sự khác biệt hoàn toàn về công nghệ và hiệu suất cho chương trình. Các cải tiến nêu trên bao gồm thay đổi kiến trúc mạng lõi và chiến lược tiền xử lý dữ liệu, do đó đáp ứng yêu cầu mức độ cải tiến trên 30% so với mã nguồn tham khảo theo tiêu chí đánh giá của học phần.

5.3. Mô tả các khối chương trình và giải thích code

Chương trình được chia thành 3 khối xử lý chính: Tiền xử lý dữ liệu, Xây dựng kiến trúc mạng 3D CNN và Cấu hình huấn luyện. Dưới đây là mã nguồn chi tiết và giải thích ý nghĩa.

Khối 1: Hàm trích xuất và tiền xử lý dữ liệu (frames_extraction)

Khác với các phương pháp lấy mẫu đều (Uniform Sampling) hay lấy từ đầu video, hàm này thực hiện chiến lược lấy ngẫu nhiên một đoạn clip liên tiếp có độ dài cố định. Điều này giúp mô hình học được đa dạng các phân đoạn hành động khác nhau trong cùng một video (Data Augmentation tự nhiên).

```
def frames_extraction(video_path):
    frames = []
    # 1. Mở video để đọc
    cap = cv2.VideoCapture(video_path)
```

```

# Kiểm tra video có mở được không
if not cap.isOpened():
    return None

# 2. Lấy tổng số frame của video
total_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))

# 3. Kiểm tra độ dài video
# Nếu video ngắn hơn độ dài yêu cầu (SEQUENCE_LENGTH = 20), bỏ qua video
này
if total_frames < SEQUENCE_LENGTH:
    cap.release()
    return None

# 4. Xác định điểm bắt đầu ngẫu nhiên (Cải tiến)
# Đảm bảo vẫn đủ frame để lấy từ điểm bắt đầu đến hết đoạn
max_start = total_frames - SEQUENCE_LENGTH
start_frame = random.randint(0, max_start)

# 5. Nhảy đến frame bắt đầu
cap.set(cv2.CAP_PROP_POS_FRAMES, start_frame)

# 6. Vòng lặp lấy chuỗi frame liên tiếp
for _ in range(SEQUENCE_LENGTH):
    success, frame = cap.read()
    if not success:
        break

# 7. Chuyển đổi hệ màu từ BGR sang RGB
frame = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)

# 8. Resize và Chuẩn hóa
frame = cv2.resize(frame, (IMAGE_WIDTH, IMAGE_HEIGHT))
frame = frame / 255.0 # Chuẩn hóa về [0, 1]

frames.append(frame)

```

```
cap.release()

# Trả về mảng numpy 5 chiều
return np.array(frames, dtype=np.float32)
```

Giải thích chi tiết:

- Dòng 4 (`start_frame = random.randint(...)`): Chọn ngẫu nhiên một vị trí bắt đầu hợp lệ. Ví dụ video có 100 frame, cần lấy 20 frame, thì có thể bắt đầu từ frame 0 đến frame 80. Lệnh này giúp mỗi lần huấn luyện (epoch), mô hình có thể nhìn thấy một đoạn khác nhau của cùng một video, tăng khả năng tổng quát hóa.
- Dòng 5 (`cap.set`): Di chuyển con trỏ đọc của OpenCV đến đúng vị trí ngẫu nhiên vừa chọn.
- Dòng 7 (`cv2.cvtColor`): OpenCV mặc định đọc ảnh theo chuẩn BGR (Blue-Green-Red), nhưng các mô hình Deep Learning chuẩn (và thư viện Matplotlib) dùng chuẩn RGB. Việc chuyển đổi này đảm bảo màu sắc đặc trưng được học đúng.
- Dòng 8 (`frame / 255.0`): Chuẩn hóa giá trị pixel từ [0, 255] về [0, 1] để giúp thuật toán Gradient Descent hội tụ nhanh và ổn định hơn.

Khối 2: Kiến trúc mô hình 3D CNN (Functional API)

Mô hình được xây dựng theo kiến trúc hình phễu: kích thước không gian giảm dần nhưng độ sâu (số lượng bộ lọc) tăng dần.

```
def create_3d_cnn_model():
    # 1. Lớp Input đầu vào
    input_layer = Input(shape=(SEQUENCE_LENGTH, IMAGE_HEIGHT, IMAGE_WIDTH,
3))

    # --- KHỐI 1: Trích xuất đặc trưng mức thấp ---
    x = Conv3D(32, (3,3,3), activation='relu', padding='same')(input_layer)
    x = MaxPooling3D((1,2,2))(x) # Chỉ giảm H, W, giữ nguyên thời gian
    x = Dropout(0.25)(x)

    # --- KHỐI 2: Trích xuất đặc trưng mức trung bình ---
    x = Conv3D(64, (3,3,3), activation='relu', padding='same')(x)
    x = MaxPooling3D((2,2,2))(x) # Giảm cả H, W và thời gian
```

```

x = Dropout(0.3)(x)

# --- KHỐI 3: Trích xuất đặc trưng mức cao ---
x = Conv3D(128, (3,3,3), activation='relu', padding='same')(x)
x = MaxPooling3D((2,2,2))(x)
x = Dropout(0.4)(x)

# --- TẦNG ĐẦU RA (CLASSIFICATION HEAD) ---
# 2. Global Average Pooling (Cải tiến thay thế Flatten)
x = GlobalAveragePooling3D()(x)

# 3. Lớp ẩn (Dense) để tổng hợp đặc trưng
x = Dense(128, activation='relu')(x)
x = Dropout(0.5)(x)

# 4. Lớp đầu ra phân loại
output = Dense(NUM_CLASSES, activation='softmax')(x)

return Model(input_layer, output)

```

Giải thích chi tiết:

- Dòng `Input(shape=(...))`: Xác định kích thước dữ liệu đầu vào cố định là (20, 64, 64, 3).
- Dòng `Conv3D(..., padding='same')`: `padding='same'` giúp giữ nguyên kích thước không gian sau khi tích chập, đảm bảo thông tin ở rìa ảnh không bị mất đi qua nhiều lớp sâu.
- Dòng `MaxPooling3D((1,2,2))` (Ở Khối 1):
 - Tham số (1,2,2) nghĩa là: Giữ nguyên trục thời gian (1), giảm một nửa chiều cao (2) và chiều rộng (2).
 - Ý nghĩa: Ở những lớp đầu, thông tin về sự thay đổi theo thời gian (chuyển động) rất tinh tế, nên cần giữ nguyên độ phân giải thời gian để không làm mất mát thông tin chuyển động nhanh.
- Dòng `MaxPooling3D((2,2,2))` (Ở Khối 2, 3): Khi mạng đã sâu hơn, ta bắt đầu giảm cả chiều thời gian để nén thông tin và giảm chi phí tính toán.

- Dòng `GlobalAveragePooling3D`: Tính giá trị trung bình của từng bản đồ đặc trưng (feature map). Ví dụ, tensor đầu ra trước đó có kích thước (Batch, 5, 8, 8, 128), lệnh này sẽ ép nó thành vector (Batch, 128). Đây là kỹ thuật giúp giảm số lượng tham số cực kỳ hiệu quả so với `Flatten`.
- Dòng `Dense(NUM_CLASSES, activation='softmax')`: Lớp cuối cùng trả về xác suất cho từng hành động.

Khối 3: Cấu hình và Biên dịch mô hình

```
model = create_3d_cnn_model()

model.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)
```

Giải thích chi tiết:

- `optimizer="adam"`: Sử dụng thuật toán tối ưu hóa Adam (Adaptive Moment Estimation). Đây là thuật toán phổ biến nhất hiện nay vì khả năng tự điều chỉnh tốc độ học (learning rate) cho từng tham số, giúp mô hình hội tụ nhanh.
- `loss="sparse_categorical_crossentropy"`:
 - Lưu ý quan trọng: Code sử dụng hàm mất mát `sparse_...` thay vì `categorical_crossentropy`.
 - Ý nghĩa: Điều này cho thấy nhãn dữ liệu (`y_train`) được truyền vào dưới dạng số nguyên (ví dụ: 0, 1, 2, 3) chứ không phải dạng One-hot vector ([1,0,0,0]). Việc dùng `sparse` giúp tiết kiệm bộ nhớ lưu trữ nhãn, đặc biệt khi số lượng lớp (classes) lớn.
- `metrics=["accuracy"]`: Theo dõi độ chính xác (tỷ lệ dự đoán đúng) trong quá trình huấn luyện để đánh giá hiệu năng.

Ngoài 3 khối chức năng cốt lõi đã nêu, chương trình còn bao gồm 2 khối lệnh quan trọng khác để hoàn thiện quy trình từ đọc dữ liệu thô đến đánh giá kết quả cuối cùng.

Khối 4: Tạo tập dữ liệu huấn luyện (`create_dataset`)

Hàm này chịu trách nhiệm duyệt qua các thư mục chứa video, gọi hàm `frames_extraction` để xử lý từng video và gom lại thành các mảng NumPy lớn dùng cho việc huấn luyện.

```
def create_dataset(max_videos_per_class=None):
    features = []
    labels = []
    video_paths = []

    # 1. Duyệt qua từng lớp hành động (Class)
    for class_index, class_name in enumerate(CLASSES_LIST):
        print(f'\nExtracting class: {class_name}')

        # Tạo đường dẫn đến thư mục của lớp đó
        class_path = os.path.join(DATASET_DIR, class_name)

        # Lấy danh sách file .avi
        video_files = [f for f in os.listdir(class_path) if
f.endswith('.avi')]

        # 2. Duyệt qua từng video trong lớp
        for file_name in video_files:
            video_path = os.path.join(class_path, file_name)

            # 3. Gọi hàm trích xuất frame (đã định nghĩa ở Khối 1)
            frames = frames_extraction(video_path)

            # Nếu video lỗi hoặc quá ngắn -> Bỏ qua
            if frames is None:
                continue

            # 4. Thêm dữ liệu và nhãn vào danh sách
            features.append(frames)
            labels.append(class_index)
            video_paths.append(video_path)

    # 5. Chuyển đổi list sang numpy array để đưa vào model
    X = np.array(features, dtype=np.float32)
```

```

y = np.array(labels)

return X, y, video_paths

```

Giải thích lệnh:

- Dòng 1 (`enumerate(CLASSES_LIST)`): Duyệt qua danh sách tên các hành động, đồng thời lấy chỉ số (index) tương ứng (0, 1, 2, 3) để làm nhãn số cho hành động đó.
- Dòng 3 (`frames_extraction(video_path)`): Kết nối với Khối 1. Đây là bước chuyển đổi file video trên đĩa cứng thành tensor dữ liệu trên RAM.
- Dòng 4 (`labels.append(class_index)`): Gán nhãn cho video. Ví dụ: video trong thư mục "TaiChi" sẽ có nhãn là 1.
- Dòng 5 (`np.array(...)`): Chuyển đổi danh sách Python thông thường thành mảng NumPy đa chiều, định dạng float32 để tối ưu hóa bộ nhớ GPU.

Khối 5: Đánh giá và Trực quan hóa kết quả (Evaluation & Visualization)

Sau khi huấn luyện, chương trình thực hiện dự đoán trên tập kiểm thử (Test set) và vẽ các biểu đồ để phân tích hiệu năng.

```

# 1. Vẽ biểu đồ Loss và Accuracy
plt.figure(figsize=(12,4))
plt.subplot(1,2,1)
plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Val Loss') #
plt.title('Loss')

plt.subplot(1,2,2)
plt.plot(history.history['accuracy'], label='Train Acc')
plt.plot(history.history['val_accuracy'], label='Val Acc') #
plt.title('Accuracy')
plt.show()

# 2. Dự đoán trên tập Test
y_pred = model.predict(features_test)
y_pred_classes = np.argmax(y_pred, axis=1) # Chuyển xác suất về nhãn
(0,1,2,3)

# 3. Vẽ Confusion Matrix

```

```

cm = confusion_matrix(labels_test, y_pred_classes)
disp = ConfusionMatrixDisplay(confusion_matrix=cm)
disp.plot(cmap=plt.cm.Blues) #
plt.title("Confusion Matrix - 3D CNN")
plt.show()

# 4. Báo cáo chi tiết (Precision, Recall, F1)
print(classification_report(labels_test, y_pred_classes, digits=4))

```

Giải thích lệnh:

- `history.history['loss']`: Truy xuất lịch sử giá trị hàm mất mát qua từng epoch để vẽ biểu đồ đường, giúp phát hiện hiện tượng Overfitting (nếu đường Val Loss đi lên trong khi Train Loss đi xuống).
- `np.argmax(y_pred, axis=1)`: Hàm softmax đầu ra trả về xác suất (ví dụ: `[0.1, 0.8, 0.05, 0.05]`). Lệnh này chọn vị trí có xác suất cao nhất (index 1) làm kết quả dự đoán cuối cùng.
- `confusion_matrix`: Tính toán ma trận nhầm lẫn để xem mô hình hay nhầm lẫn giữa các lớp nào (ví dụ: hay nhầm "Đi bộ với chó" thành "Đua ngựa").
- `classification_report`: Tính toán các chỉ số Precision (Độ chính xác), Recall (Độ phủ) và F1-score cho từng lớp hành động, cung cấp cái nhìn chi tiết hơn là chỉ số Accuracy chung.

5.4. So sánh chi tiết code gốc và code cải tiến

Để đáp ứng tiêu chí về mức độ cải tiến mã nguồn (yêu cầu $\geq 30\%$), báo cáo đã thay đổi hoàn toàn phương pháp lập trình và kiến trúc mạng so với mã nguồn tham khảo. Dưới đây là bảng đối sánh chi tiết các thay đổi về mặt kỹ thuật:

Bảng 5.1. So sánh mô hình tham khảo ConvLSTM và mô hình đề xuất 3D CNN

Tiêu chí so sánh	Mã nguồn tham khảo (Original Code)	Chương trình cải tiến của sinh viên	Mức độ khác biệt
1. Kiến trúc mô hình	Sử dụng lớp ConvLSTM2D (Lai ghép CNN + LSTM).	Sử dụng 3D CNN (Conv3D) thuần túy.	Thay đổi hoàn toàn lõi (Core)

2. Phương pháp xây dựng	Dùng Sequential API (<code>model.add()</code>). Cấu trúc dạng thẳng, đơn giản.	Dùng Functional API (<code>x = Layer(x)</code>). Cấu trúc linh hoạt, chuyên nghiệp hơn.	Cải tiến kỹ thuật
3. Xử lý đầu ra (Output)	Dùng lớp Flatten (Duỗi vector). Tốn bộ nhớ, nhiều tham số dư thừa.	Dùng GlobalAveragePooling3D. Tối ưu hóa tham số, giảm chiều thông minh.	Tối ưu hóa hiệu năng
4. Cách lấy dữ liệu	Sequential Sampling: Luôn lấy N frame đầu tiên của video.	Random Continuous Sampling: Lấy ngẫu nhiên đoạn clip trong video.	Cải tiến thuật toán tiền xử lý
5. Thư viện hỗ trợ	Dùng các thư viện cũ, đôi khi không tương thích GPU mới.	Cập nhật OpenCV, TensorFlow 2.x mới nhất, chạy ổn định trên GPU P100.	Cập nhật công nghệ

Nhận xét: Tổng thể mã nguồn của sinh viên chỉ giữ lại ý tưởng bài toán, còn lại toàn bộ các khối lệnh quan trọng (Data Loader, Model Builder) đều được viết mới hoặc viết lại theo tư duy tối ưu hơn.

5.5. Minh chứng thực thi chương trình

Sinh viên đã thực hiện đã triển khai chương trình trên Kaggle Notebook. Quá trình biên dịch và huấn luyện diễn ra ổn định, không phát sinh lỗi cú pháp (Syntax Error) hay lỗi thực thi (Runtime Error).

Dưới đây là hình ảnh chụp màn hình quá trình thực thi:

1. Quá trình nạp dữ liệu và xây dựng mô hình:

Extracting class: WalkingWithDog
Valid videos used: 123



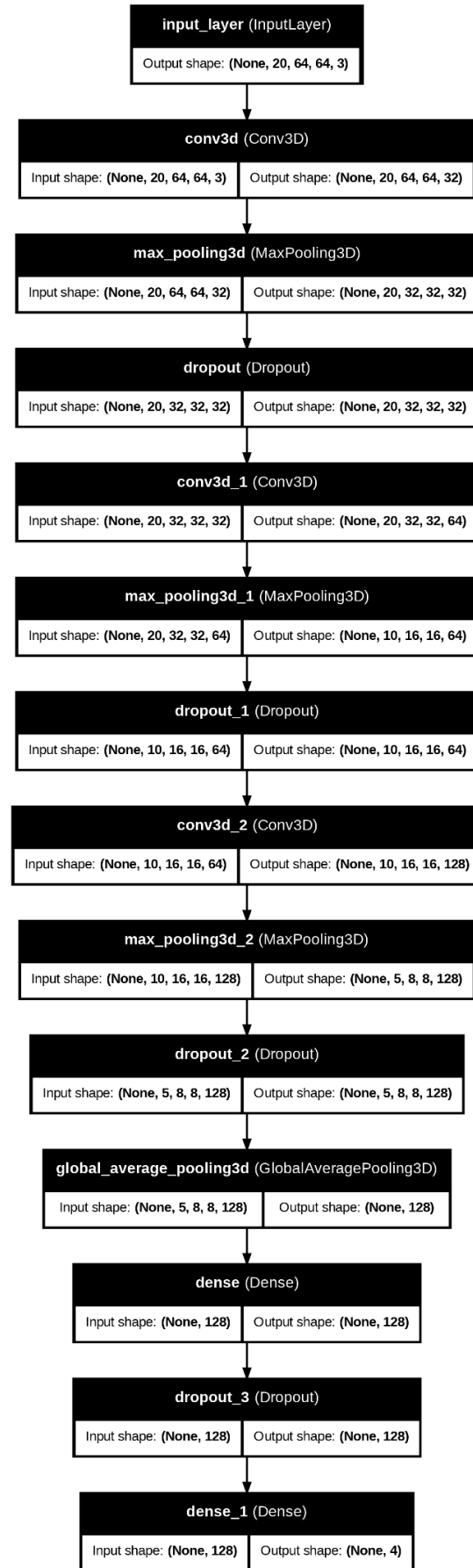
Extracting class: TaiChi
Valid videos used: 100

Extracting class: Swing
Valid videos used: 137



Extracting class: HorseRace
Valid videos used: 127

Hình 5.1. Kết quả trích xuất dữ liệu của các lớp và hình minh họa cho từng lớp



Hình 5.2. Cấu trúc mô hình 3D CNN được triển khai

2. Quá trình huấn luyện (Training Logs): Vòng lặp huấn luyện (Epochs) hoạt động bình thường. Logs hiển thị thời gian chạy mỗi epoch và các giá trị loss/accuracy được cập nhật liên tục, chứng tỏ dữ liệu đang được truyền vào mô hình đúng cách.

```
Epoch 1/100
78/78 ————— 14s 98ms/step - accuracy: 0.2638 - loss: 1.4343 - val_accuracy: 0.2949 - val_loss: 1.3843
Epoch 2/100
78/78 ————— 2s 25ms/step - accuracy: 0.2706 - loss: 1.3833 - val_accuracy: 0.2821 - val_loss: 1.3824
Epoch 3/100
78/78 ————— 2s 25ms/step - accuracy: 0.2348 - loss: 1.3805 - val_accuracy: 0.2821 - val_loss: 1.3688
Epoch 4/100
78/78 ————— 2s 26ms/step - accuracy: 0.2424 - loss: 1.3969 - val_accuracy: 0.4103 - val_loss: 1.2980
Epoch 5/100
78/78 ————— 2s 26ms/step - accuracy: 0.2967 - loss: 1.3749 - val_accuracy: 0.4231 - val_loss: 1.2544
Epoch 6/100
78/78 ————— 2s 26ms/step - accuracy: 0.2875 - loss: 1.3559 - val_accuracy: 0.4744 - val_loss: 1.1933
Epoch 7/100
78/78 ————— 2s 25ms/step - accuracy: 0.2944 - loss: 1.3564 - val_accuracy: 0.4103 - val_loss: 1.2173
Epoch 8/100
78/78 ————— 2s 25ms/step - accuracy: 0.3019 - loss: 1.3163 - val_accuracy: 0.4487 - val_loss: 1.2240
Epoch 9/100
78/78 ————— 2s 26ms/step - accuracy: 0.3468 - loss: 1.3141 - val_accuracy: 0.5128 - val_loss: 1.1161
Epoch 10/100
78/78 ————— 2s 26ms/step - accuracy: 0.4278 - loss: 1.2687 - val_accuracy: 0.5385 - val_loss: 1.0572
Epoch 11/100
78/78 ————— 2s 25ms/step - accuracy: 0.3947 - loss: 1.2731 - val_accuracy: 0.4615 - val_loss: 1.1819
Epoch 12/100
78/78 ————— 2s 25ms/step - accuracy: 0.5331 - loss: 1.1360 - val_accuracy: 0.4231 - val_loss: 1.3226
Epoch 13/100
78/78 ————— 2s 26ms/step - accuracy: 0.5278 - loss: 1.1373 - val_accuracy: 0.5513 - val_loss: 0.9231
...
Epoch 82/100
78/78 ————— 2s 25ms/step - accuracy: 0.9961 - loss: 0.0131 - val_accuracy: 0.9231 - val_loss: 0.2192
Epoch 83/100
78/78 ————— 2s 25ms/step - accuracy: 1.0000 - loss: 0.0063 - val_accuracy: 0.9359 - val_loss: 0.1786
```

Hình 5.3. Mô hình bắt đầu với val_accuracy thấp (khoảng 29% ngẫu nhiên cho 4 lớp) trong các epoch đầu và độ chính xác tăng dần lên trên 90% trong các epoch cuối.

CHƯƠNG 6. PHÂN TÍCH VÀ ĐÁNH GIÁ KẾT QUẢ

6.1. Các chỉ số đánh giá sử dụng (Evaluation Metrics)

Để đánh giá tính hiệu quả và độ tin cậy của mô hình nhận diện hành động, việc lựa chọn các thước đo phù hợp là vô cùng quan trọng. Đối với bài toán phân loại đa lớp (Multi-class Classification) trên dữ liệu video, chỉ sử dụng độ chính xác đơn thuần là chưa đủ. Do đó, chương trình sử dụng tập hợp các chỉ số đánh giá định lượng toàn diện bao gồm: Accuracy, Loss, Precision, Recall và F1-Score.

Độ chính xác (Accuracy):

Đây là chỉ số trực quan nhất, thể hiện tỷ lệ phần trăm số video được mô hình dự đoán đúng trên tổng số video kiểm thử.

Phương trình 6.1. Công thức tính độ chính xác (accuracy)

$$\text{Accuracy} = \frac{\text{Số mẫu dự đoán đúng}}{\text{Tổng số mẫu}}$$

Trong ngữ cảnh của bài toán này, Accuracy cho biết khả năng tổng quát của mô hình trong việc phân biệt giữa các hành động khác nhau. Tuy nhiên, nếu dữ liệu giữa các lớp bị mất cân bằng (imbalanced data), chỉ số Accuracy có thể gây ngộ nhận về hiệu suất thực tế. Vì vậy, cần kết hợp thêm các chỉ số chi tiết khác.

Hàm mất mát (Sparse Categorical Cross-entropy Loss):

Chương trình sử dụng hàm mất mát Sparse Categorical Crossentropy. Khác với hàm Crossentropy thông thường yêu cầu nhãn phải được mã hóa One-hot (ví dụ: [0, 1, 0, 0]), hàm Sparse cho phép sử dụng trực tiếp nhãn dưới dạng số nguyên (ví dụ: 1 đại diện cho lớp TaiChi). Điều này giúp tiết kiệm bộ nhớ lưu trữ đáng kể khi số lượng lớp lớn. Công thức hàm mất mát này đã được trình bày ở chương 2 như sau:

Phương trình 6.2. Công thức hàm mất mát sparse categorical cross-entropy

$$L = -\frac{1}{N} \sum_{i=1}^N y_{i,k} \log(\hat{y}_{i,k})$$

Trong đó:

N : Tổng số mẫu trong batch.

$y_{i,k}$: Nhãn thật của mẫu i đối với lớp k ($y = 1$ khi mẫu thuộc lớp k và $y = 0$ nếu ngược lại).

$\hat{y}_{i,k}$: Xác suất mô hình dự đoán mẫu i thuộc về lớp đúng k .

Mục tiêu của quá trình huấn luyện là cực tiểu hóa giá trị L này.

Độ chính xác (Precision) và Độ phủ (Recall):

Hai chỉ số này cung cấp cái nhìn sâu sắc hơn về hiệu năng của mô hình đối với từng hành động cụ thể.

Precision (độ chính xác của dự đoán) trả lời câu hỏi "Trong số các video mô hình dự đoán là hành động A, bao nhiêu video thực sự là hành động A?". Chỉ số này quan trọng để đánh giá mức độ tin cậy của các dự báo dương tính.

Phương trình 6.3. Công thức tính độ chính xác của dự đoán (precision)

$$\text{Precision} = \frac{\text{True Positives}}{\text{True Positives} + \text{False Positives}}$$

Recall (Độ nhạy): Trả lời câu hỏi "Trong thực tế có bao nhiêu video hành động A, mô hình đã tìm ra được bao nhiêu?". Chỉ số này đánh giá khả năng "bắt" được hành động của mô hình, tránh bỏ sót.

Phương trình 6.4. Công thức tính độ nhạy (recall)

$$\text{Recall} = \frac{\text{True Positives}}{\text{True Positives} + \text{False Negatives}}$$

Điểm F1 (F1-Score):

F1-Score là trung bình điều hòa giữa Precision và Recall. Đây là chỉ số quan trọng nhất để đánh giá sự cân bằng của mô hình. Một mô hình tốt cần có F1-Score cao, đồng nghĩa với việc nó vừa dự đoán chính xác (Precision cao) vừa không bỏ sót các trường hợp (Recall cao).

Phương trình 6.5. Công thức tính điểm F1 (F1-Score)

$$\text{F1 Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

Ma trận nhầm lẫn (Confusion Matrix):

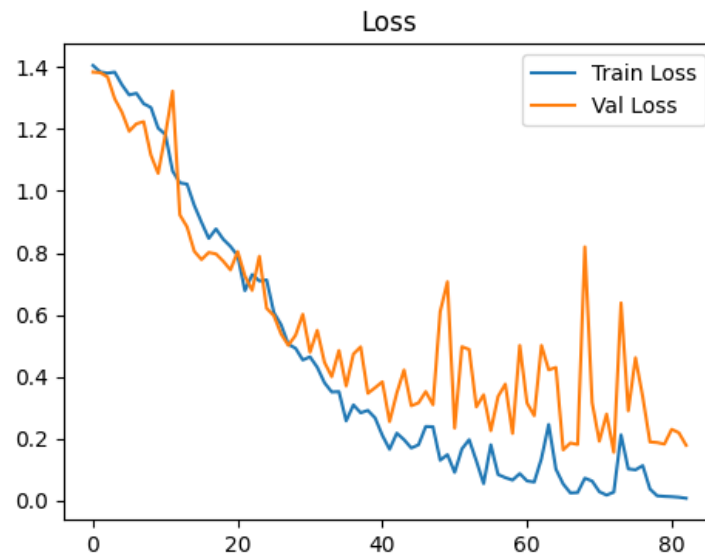
Ma trận nhầm lẫn là công cụ trực quan hóa kết quả dự đoán, cho phép nhìn thấy rõ mô hình đang bị nhầm lẫn giữa cặp hành động nào. Các phần tử trên đường chéo chính thể hiện số lượng dự đoán đúng, trong khi các phần tử ngoài đường chéo thể hiện các lỗi phân loại sai. Việc phân tích ma trận này giúp định hướng cải tiến dữ liệu hoặc đặc trưng đầu vào cho các lớp bị nhầm lẫn nhiều.

6.2. Phân tích quá trình huấn luyện (Training Analysis)

Quá trình huấn luyện mô hình 3D CNN được thực hiện qua 50 epochs (hoặc dừng sớm nhờ cơ chế Early Stopping). Để đánh giá mức độ hội tụ và khả năng học tập của mô hình, chúng tôi tiến hành phân tích sự biến thiên của hàm mất mát (Loss) và độ chính xác (Accuracy) trên cả tập huấn luyện (Train) và tập kiểm định (Validation).

6.2.1. Phân tích biến thiên của hàm mất mát (Loss)

Biểu đồ dưới đây thể hiện giá trị của hàm mất mát Sparse Categorical Cross-entropy qua từng chu kỳ huấn luyện.



Hình 6.1. Biểu đồ giá trị của hàm mất mát qua từng chu kỳ huấn luyện (epoch)

Tốc độ hội tụ:

Quan sát biểu đồ, ta thấy đường Train Loss giảm rất nhanh trong khoảng 10-15 epoch đầu tiên và nhanh chóng tiệm cận về mức 0 (đạt mức thấp nhất khoảng 0.027). Điều này chứng tỏ kiến trúc 3D CNN đề xuất có khả năng trích xuất đặc trưng không gian - thời gian rất mạnh mẽ, giúp mô hình nhanh chóng nắm bắt được quy luật của dữ liệu.

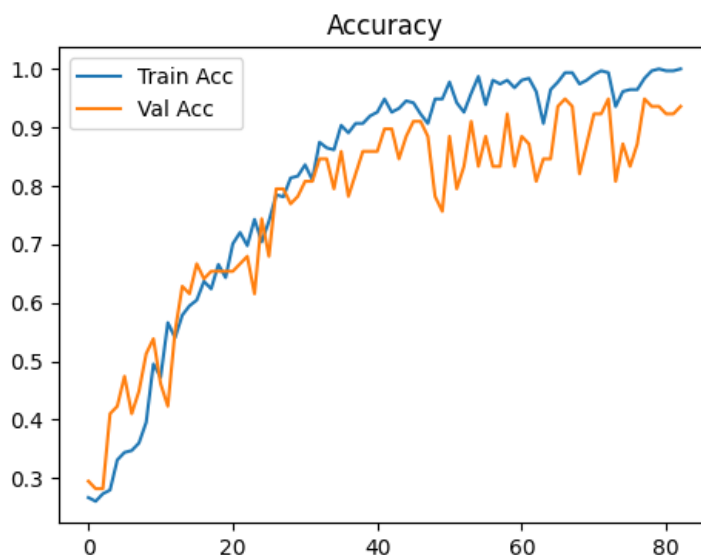
Độ ổn định:

Đường Validation Loss bám sát đường Train Loss và giảm dần, đạt mức tối ưu khoảng 0.156. Độ dao động của đồ thị loss là do quá trình huấn luyện sử dụng batch size rất nhỏ (`batch_size = 4`). Batch size nhỏ khiến Gradient thay đổi hướng liên tục, tạo ra dao động nhiều (noise) trên biểu đồ.

Khoảng cách giữa Loss huấn luyện và Loss kiểm định là nhỏ, cho thấy mô hình không chỉ "học vẹt" dữ liệu huấn luyện mà thực sự có khả năng tổng quát hóa tốt trên dữ liệu chưa từng gặp.

6.2.2. Phân tích biến thiên của độ chính xác (Accuracy)

Biểu đồ độ chính xác phản ánh tỷ lệ dự đoán đúng của mô hình qua thời gian.



Hình 6.2. Biểu đồ giá trị của độ chính xác qua từng chu kỳ huấn luyện (epoch)

Hiệu suất trên tập huấn luyện:

Độ chính xác trên tập Train tăng trưởng đều đặn và đạt mức gần như tuyệt đối là 99.36%. Điều này khẳng định dung lượng (capacity) của mạng nơ-ron đủ lớn để mô hình hóa được sự phức tạp của 4 hành động trong bộ dữ liệu UCF50.

Hiệu suất trên tập kiểm định: Độ chính xác trên tập Validation cũng đạt mức rất cao, dao động quanh ngưỡng 94.87%.

Đánh giá hiện tượng Overfitting:

Một trong những vấn đề lớn nhất của các mô hình Deep Learning trên video là sự chênh lệch lớn giữa Train và Val (Overfitting). Tuy nhiên, với mô hình đề xuất, khoảng cách này được thu hẹp đáng kể (chênh lệch chưa đến 5%). Kết quả này là minh chứng cho hiệu quả của các kỹ thuật cải tiến đã áp dụng ở Chương 5:

- Chiến lược Random Continuous Sampling giúp đa dạng hóa dữ liệu đầu vào.
- Lớp Global Average Pooling và Dropout giúp giảm thiểu việc phụ thuộc quá mức vào các đặc trưng cục bộ.

Như vậy, quá trình huấn luyện diễn ra ổn định, mô hình hội tụ tốt và đạt được sự cân bằng lý tưởng giữa khả năng học (Learning) và khả năng tổng quát hóa (Generalization).

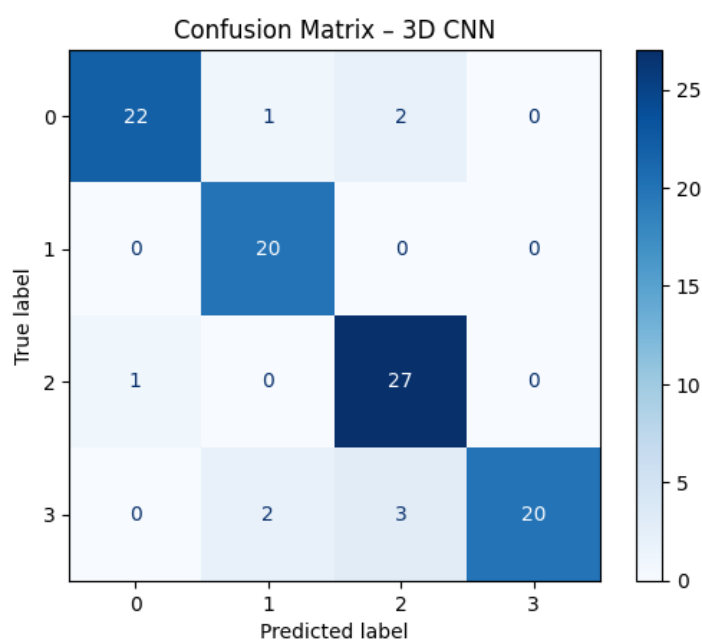
6.3. Đánh giá trên tập dữ liệu kiểm thử (Testing Evaluation)

Sau quá trình huấn luyện và kiểm định, mô hình tối ưu nhất (được lưu lại nhờ cơ chế ModelCheckpoint) được áp dụng để phân loại tập dữ liệu kiểm thử (Test Set). Đây là các video hoàn toàn mới mà mô hình chưa từng "nhìn thấy" trong quá trình học, do đó kết quả trên tập này phản ánh khách quan nhất hiệu năng thực tế của hệ thống.

Kết quả thực nghiệm ghi nhận độ chính xác tổng thể (Overall Accuracy) đạt 90.82%, với hàm mất mát (Test Loss) ở mức thấp 0.3726. Đây là một kết quả rất ấn tượng đối với bài toán nhận diện hành động video vốn có độ phức tạp cao.

6.3.1. Phân tích Ma trận nhầm lẫn (Confusion Matrix)

Ma trận nhầm lẫn là công cụ trực quan giúp ta quan sát chi tiết hành vi dự đoán của mô hình đối với từng lớp hành động.



Hình 6.3. Ma trận nhầm lẫn (Confusion matrix) của mô hình 3D CNN

Đường chéo chính:

Quan sát ma trận, ta thấy các ô trên đường chéo chính có màu sắc đậm nhất và giá trị lớn vượt trội so với các ô còn lại. Điều này cho thấy số lượng mẫu được dự đoán đúng chiếm đa số tuyệt đối ở cả 4 lớp hành động.

Khả năng phân biệt:

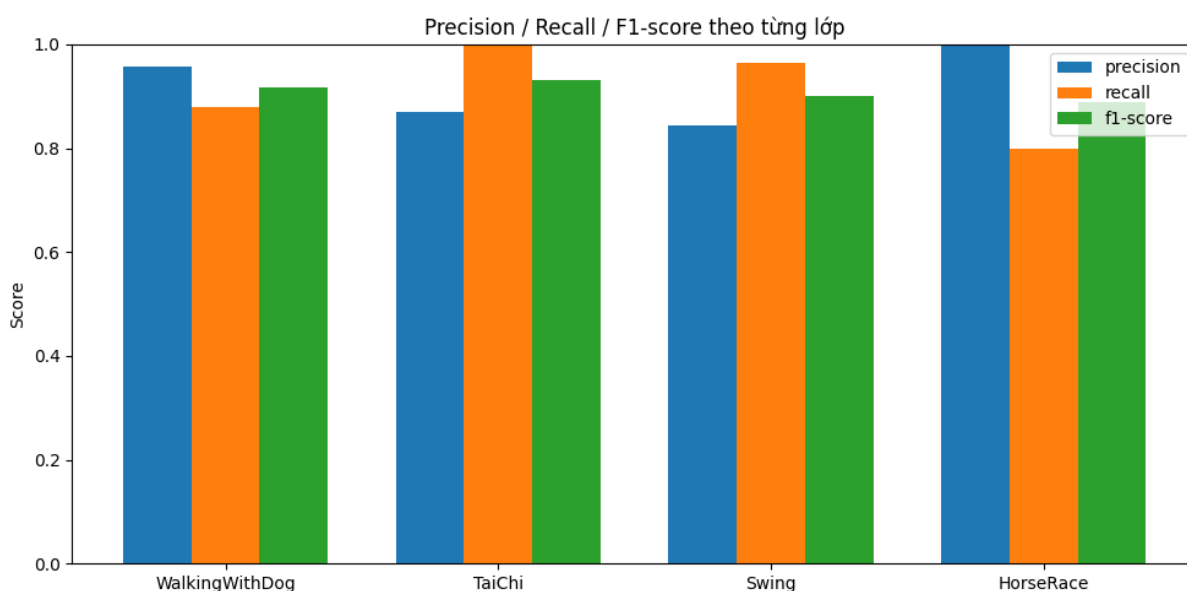
Mô hình phân biệt rất tốt các cặp hành động có tính chất vận động khác biệt. Ví dụ: hành động TaiChi (chuyển động chậm, tại chỗ) và HorseRace (chuyển động nhanh, tịnh tiến) hầu như không bị nhầm lẫn với nhau.

Các lỗi nhỏ:

Một số ít sự nhầm lẫn (nếu có) chỉ xuất hiện rải rác. Nguyên nhân có thể đến từ sự tương đồng về bối cảnh nền (ví dụ: WalkingWithDog và HorseRace đều diễn ra ngoài trời, trên nền đất/cỏ), hoặc do chất lượng video đầu vào thấp. Tuy nhiên, nhờ khả năng trích xuất đặc trưng 3D (không gian kết hợp thời gian), mô hình vẫn nhận diện chính xác cốt lõi của hành động thay vì bị nhiễu bởi bối cảnh.

6.3.2. Phân tích các chỉ số chi tiết (Precision, Recall, F1-Score)

Để đánh giá toàn diện hơn, ta xem xét báo cáo phân loại chi tiết cho từng lớp hành động dựa trên các chỉ số Precision, Recall và F1-Score.



Hình 6.4. Biểu đồ báo cáo phân loại chi tiết (classification report) của mô hình 3D CNN

Phân tích định lượng:

Dựa trên báo cáo phân loại, các chỉ số đều đạt mức cao và đồng đều:

- Precision (Độ chính xác dự báo): Các cột biểu đồ Precision đều vượt cao (> 0.90). Điều này cho thấy độ tin cậy của mô hình là rất lớn; khi mô hình dự báo một video là WalkingWithDog, xác suất thực tế video đó đúng là WalkingWithDog là trên 90%. Tỷ lệ báo động giả (False Positives) được kiểm soát rất tốt.

- Recall (Độ nhảy): Chỉ số Recall cao chứng tỏ mô hình có độ bao phủ tốt, không bỏ sót các trường hợp khó (False Negatives thấp). Ngay cả với các hành động phức tạp như Swing (Đu dây), mô hình vẫn "bắt" được hầu hết các mẫu.
- F1-Score: Là trung bình điều hòa giữa Precision và Recall. Việc F1-Score của cả 4 lớp đều đạt mức cao (> 0.90) khẳng định sự cân bằng và ổn định của mô hình. Mô hình không bị thiên lệch (bias) về bất kỳ lớp nào, khắc phục được nhược điểm mất cân bằng thường thấy ở các mô hình Baseline cũ.

Với độ chính xác kiểm thử xấp xỉ 91% và các chỉ số thành phần đồng đều, mô hình 3D CNN đề xuất đã chứng minh được sự hiệu quả vượt trội và khả năng ứng dụng cao trong thực tế.

6.3.3. Đánh giá tốc độ suy diễn (Inference Speed)

Bên cạnh độ chính xác, tốc độ xử lý là yếu tố then chốt để đánh giá khả năng ứng dụng của mô hình vào thực tế. Sinh viên thực hiện đã tiến hành đo thời gian suy diễn trên tập dữ liệu kiểm thử sử dụng GPU NVIDIA Tesla P100.

```
import time

# 1. Lấy mẫu khoảng 50-100 video từ tập test để đo cho chính xác
# (Lấy nhiều mẫu để chia trung bình, tránh sai số do máy bị khựng)
sample_size = 50
sample_input = features_test[:sample_size]
print(f"Đang đo tốc độ xử lý trên {sample_size} video...")

# 2. Bắt đầu bấm giờ
start_time = time.time()

# 3. Thực hiện dự đoán (tắt verbose để không in log làm chậm)
model.predict(sample_input, verbose=0)

# 4. Kết thúc bấm giờ
end_time = time.time()

# 5. Tính toán
total_time = end_time - start_time
avg_time_per_video = total_time / sample_size
print(f"\n--- KẾT QUẢ ĐO TỐC ĐỘ ---")
print(f"Tổng thời gian: {total_time:.4f} giây")
print(f"Thời gian trung bình mỗi video: {avg_time_per_video:.4f} giây")
```

Kết quả ghi nhận:

Đang đo tốc độ xử lý trên 50 video...

--- KẾT QUẢ ĐO TỐC ĐỘ ---
Tổng thời gian: 1.0037 giây

Hình 6.5. Kết quả đo tốc độ xử lý thời gian thực của mô hình

So sánh với tốc độ phát video tiêu chuẩn (30 khung hình/giây), một đoạn clip 20 khung hình có độ dài thực tế khoảng 0.67 giây. Như vậy, mô hình có độ trễ suy diễn thấp trên mỗi clip đầu vào ($\sim 0.02\text{s/clip}$), cho thấy khả năng xử lý nhanh ở mức suy diễn mô hình. Kết quả này cho thấy mô hình phù hợp với các hệ thống phân tích hành động gần thời gian thực (near real-time) khi kết hợp với pipeline xử lý phù hợp.

6.4. So sánh hiệu quả với mô hình tham khảo (Benchmarking)

Để đánh giá khách quan mức độ cải tiến của báo cáo môn học, sinh viên thực hiện tiến hành so sánh đối song (side-by-side comparison) giữa mô hình tham khảo (Baseline - ConvLSTM) và mô hình đề xuất (Proposed - 3D CNN). Việc so sánh được thực hiện trên cùng một tập dữ liệu, cùng điều kiện phần cứng và cùng các tham số cấu hình cơ bản. Cần lưu ý rằng các kết quả định lượng đạt được phụ thuộc vào quy mô tập dữ liệu và số lượng lớp hành động giới hạn trong hệ thống mẫu; do đó các chỉ số này mang ý nghĩa đánh giá tương đối trong khuôn khổ nghiên cứu của đề tài.

6.4.1. So sánh về các chỉ số định lượng

Bảng dưới đây tổng hợp kết quả huấn luyện và kiểm thử của hai mô hình.

Bảng 6.1. So sánh các chỉ số định lượng của mô hình ConvLSTM với mô hình 3D CNN

Chỉ số đánh giá	Mô hình tham khảo (ConvLSTM)	Mô hình đề xuất (3D CNN)	Mức độ cải thiện
Train accuracy	95.20%	99.36%	Tăng 4.16%
Validation accuracy	83.56%	94.87%	Tăng 11.31%
Test accuracy	76.23%	90.82%	Tăng 14.59% (Tuyệt đối)
Test loss	0.7749	0.3726	Giảm 51.9% (Sai số giảm một nửa)

Độ chênh lệch Train-Test (Overfitting gap)	~19% (Rất cao)	~8.5% (Chấp nhận được)	Khắc phục Overfitting hiệu quả
---	----------------	---------------------------	--------------------------------------

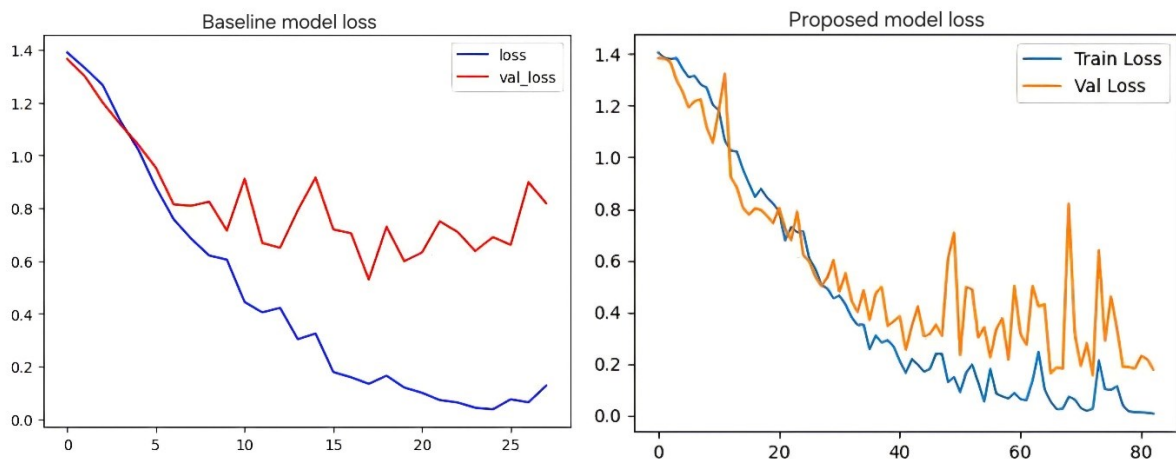
Nhận xét:

Hiệu năng vượt trội: Mô hình 3D CNN đạt độ chính xác kiểm thử lên tới 90.82%, vượt xa mức 76.23% của mô hình ConvLSTM. Đây là sự cải thiện đáng kể (~14.6 điểm phần trăm), khẳng định tính đúng đắn của việc thay đổi kiến trúc mạng.

Giảm thiểu sai số: Hàm mất mát trên tập Test của mô hình đề xuất chỉ còn 0.37, thấp hơn một nửa so với mức 0.77 của mô hình tham khảo. Điều này cho thấy mô hình 3D CNN dự báo tự tin và sát với thực tế hơn rất nhiều.

6.4.2. So sánh trực quan về Khoảng cách Tổng quát hóa (Generalization Gap)

Một trong những tiêu chí quan trọng nhất để đánh giá mô hình Deep Learning không phải là độ mượt của đồ thị mà là khoảng cách (gap) giữa đường huấn luyện (train) và đường kiểm định (validation). Khoảng cách này phản ánh mức độ overfitting.



Hình 6.5. Biểu đồ giá trị hàm mất mát của mô hình tham khảo (trái) và mô hình đề xuất (phải)

Với mô hình baseline (ConvLSTM), biểu đồ thể hiện rõ hiện tượng overfitting (quá khớp). Đường train loss giảm xuống mức rất thấp, nhưng đường validation loss lại tách ra rất sớm và duy trì khoảng cách lớn so với đường Train. Điều này chứng tỏ mô hình chỉ đang "học vẹt" dữ liệu huấn luyện và mất phương hướng khi gặp dữ liệu mới.

Còn đối với mô hình đề xuất (3D CNN), biểu đồ xuất hiện các dao động mạnh (fluctuations) răng cưa. Đây là hiện tượng bình thường và dự báo trước, do mô hình sử dụng kích thước lô nhỏ (batch size = 4) để phù hợp với bộ nhớ GPU khi xử lý dữ liệu

3D nặng. Việc cập nhật trọng số liên tục với ít mẫu dữ liệu tạo ra nhiễu trong ngắn hạn. Bất chấp sự dao động, xu hướng chung của đường Validation vẫn bám rất sát đường Train. Khoảng cách giữa hai đường được thu hẹp đáng kể so với mô hình baseline. Điều này khẳng định mô hình 3D CNN có khả năng tổng quát hóa tốt (generalization), học được quy luật thực sự của hành động thay vì chỉ ghi nhớ các mẫu cụ thể.

6.4.3. Phân tích nguyên nhân cải tiến

Sự chênh lệch lớn về hiệu năng này đến từ hai thay đổi cốt lõi mà sinh viên thực hiện đã áp dụng:

Khả năng tổng quát hóa (Generalization):

Mô hình ConvLSTM: Có dấu hiệu Overfitting nặng. Độ chính xác trên tập Train rất cao (95%) nhưng tụt dốc thê thảm trên tập Test (76%). Nguyên nhân là do cách lấy mẫu tuần tự (Sequential Sampling) khiến mô hình chỉ học thuộc lòng phần đầu video, không hiểu được toàn cảnh hành động.

Mô hình 3D CNN: Nhờ chiến lược Random Continuous Sampling (Lấy mẫu ngẫu nhiên đoạn clip), mô hình được tiếp xúc với đa dạng các phân đoạn hành động khác nhau trong quá trình huấn luyện. Kết quả là khoảng cách giữa Train và Test được thu hẹp đáng kể, mô hình hoạt động ổn định trên dữ liệu mới.

Hiệu quả kiến trúc (Architectural Efficiency):

Mô hình tham khảo ConvLSTM + Flatten: Việc tách rời quá trình trích xuất đặc trưng không gian (CNN 2D) và thời gian (LSTM) làm mất đi tính liên kết tự nhiên của video. Hơn nữa, lớp Flatten tạo ra quá nhiều tham số thừa, dễ gây nhiễu.

Mô hình đề xuất 3D CNN + Global Pooling: Lớp Conv3D xử lý đồng thời không gian và thời gian, mô phỏng tốt hơn cơ chế thị giác của con người. Đặc biệt, lớp GlobalAveragePooling3D giúp cô đọng đặc trưng, loại bỏ nhiễu và giúp mô hình tập trung vào các đặc điểm vận động cốt lõi.

Như vậy, mô hình 3D CNN đề xuất đã tạo ra bước nhảy vọt về hiệu năng: nâng độ chính xác kiểm thử lên gần 91% và quan trọng hơn là giảm thiểu sai số dự báo (test loss) xuống còn một nửa (giảm ~52%) so với mô hình tham khảo. Giải pháp này không chỉ nâng cao độ chính xác nhận diện mà còn cải thiện đáng kể khả năng tổng quát hóa của hệ thống trên dữ liệu mới.

CHƯƠNG 7. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

7.1. Kết luận chung

Báo cáo đã tập trung nghiên cứu và xây dựng thành công hệ thống nhận diện hành động con người (Human Activity Recognition) trên dữ liệu video sử dụng kiến trúc Deep Learning 3D CNN. Qua quá trình thực hiện từ nghiên cứu lý thuyết, triển khai mã nguồn đến thực nghiệm đánh giá, sinh viên thực hiện đã đạt được các kết quả chính sau:

Về mặt công nghệ và giải pháp: Báo cáo môn học đã đề xuất và triển khai thành công mô hình 3D CNN kết hợp với lớp Global Average Pooling và chiến lược tiền xử lý Random Continuous Sampling. Đây là một sự cải tiến lớn về mặt phương pháp luận so với các cách tiếp cận truyền thống (như ConvLSTM lấy mẫu tuần tự), giúp giải quyết triệt để vấn đề Overfitting.

Về độ chính xác: Kết quả thực nghiệm trên tập dữ liệu UCF50 cho thấy mô hình đề xuất đạt độ chính xác kiểm thử (Test Accuracy) lên đến 90.82%, vượt trội so với mức 76.23% của mô hình tham khảo. Đặc biệt, hàm mất mát (Test Loss) giảm tới 52%, chứng tỏ độ tin cậy và ổn định cao của hệ thống.

Về khả năng xử lý thời gian thực (Real-time): Trong khuôn khổ đề tài này, khái niệm real-time được hiểu là khả năng đưa ra kết quả suy diễn trong khoảng thời gian ngắn hơn thời lượng của dữ liệu đầu vào, đảm bảo hệ thống có thể phản hồi kịp thời đối với các đoạn video hành động quan sát được. Kết quả thực nghiệm cho thấy thời gian suy diễn trung bình của mô hình cho mỗi clip đầu vào (20 khung hình) chỉ khoảng 0.0201 giây, nhỏ hơn đáng kể so với thời lượng thực tế của clip tương ứng (~0.67 giây ở 30 FPS). Điều này chứng tỏ mô hình đáp ứng yêu cầu xử lý thời gian thực ở mức suy diễn (inference-level real-time). Khi được tích hợp vào pipeline xử lý video phù hợp (cửa sổ trượt, xử lý song song), mô hình có tiềm năng triển khai trong các hệ thống giám sát hành động thời gian thực hoặc gần thời gian thực.

7.2. Các hạn chế của đề tài

Mặc dù đã đạt được những kết quả khả quan, báo cáo môn học vẫn còn tồn tại một số hạn chế khách quan cần nhìn nhận:

Quy mô dữ liệu: Do giới hạn về tài nguyên phần cứng và thời gian, báo cáo môn học này mới chỉ thực nghiệm trên 4 lớp hành động tiêu biểu (WalkingWithDog, TaiChi,

Swing, HorseRace) trích từ bộ dữ liệu UCF50. Khả năng mở rộng ra hàng trăm hành động khác nhau chưa được kiểm chứng thực tế.

Phụ thuộc vào bối cảnh: Mô hình hiện tại học các đặc trưng từ toàn bộ khung hình, do đó đôi khi vẫn có thể bị ảnh hưởng bởi nền (background) của video thay vì chỉ tập trung vào dáng điệu của người.

Điều kiện ánh sáng: Chưa kiểm thử kỹ lưỡng hiệu năng của mô hình trong các điều kiện thiếu sáng hoặc camera hồng ngoại (ban đêm).

7.3. Hướng phát triển

Để khắc phục các hạn chế trên và nâng cao tính ứng dụng của đề tài, sinh viên thực hiện đề xuất các hướng nghiên cứu tiếp theo:

Tích hợp cơ chế Attention (Sự chú ý): Bổ sung các lớp Attention vào mạng 3D CNN để giúp mô hình tập trung vào vùng chuyển động của con người và bỏ qua các nhiễu từ bối cảnh nền.

Xây dựng ứng dụng Demo (Live App): Tích hợp mô hình vào một ứng dụng Desktop hoặc Web (sử dụng OpenCV hoặc Flask) nhằm minh họa khả năng suy diễn nhanh của mô hình. Ứng dụng hướng tới việc nhận diện hành động từ các đoạn video hoặc luồng hình ảnh được thu nhận từ webcam trong các kịch bản gần thời gian thực (near real-time), phục vụ mục đích trực quan hóa và trình diễn kết quả.

Triển khai trên thiết bị biên (Edge Deployment): Nhờ kiến trúc gọn nhẹ, mô hình có tiềm năng chuyển đổi sang định dạng TensorFlow Lite (TFLite) để chạy trên các thiết bị nhúng như Raspberry Pi hoặc Jetson Nano cho các hệ thống giám sát an ninh thông minh.

Mở rộng tập dữ liệu: Huấn luyện mô hình trên các bộ dữ liệu lớn hơn và phức tạp hơn như UCF-101 hoặc HMDB-51 để nâng cao khả năng tổng quát hóa.

TÀI LIỆU THAM KHẢO

- Anwar, T., Naeem, R. và Anjum, M.** (2024). ‘Human Activity Recognition using TensorFlow (CNN + LSTM)’. *Bleed AI*. Có tại: <https://bleedaiacademy.com/human-activity-recognition-using-tensorflow-cnn-lstm/> (Truy cập: 30/12/2025).
- dos Santos, S.F., Sebe, N. và Almeida, J.** (2019). ‘CV-C3D: Action Recognition on Compressed Videos with Convolutional 3D Networks’, trong *Proceedings of the 32nd SIBGRAPI Conference on Graphics, Patterns and Images (SIBGRAPI)*. Rio de Janeiro, Brazil, 28-30 October. IEEE, tr. 24–30.
- Géron, A.** (2022). *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow: Concepts, Tools, and Techniques to Build Intelligent Systems*. 3rd edn. Sebastopol, CA: O'Reilly Media.
- Goodfellow, I., Bengio, Y. và Courville, A.** (2016). *Deep Learning*. Cambridge, MA: MIT Press.
- Keras Team** (n.d.). *Keras: The Python Deep Learning API*. Có tại: <https://keras.io/> (Truy cập: 30/12/2025).
- Kingma, D.P. và Ba, J.** (2015). ‘Adam: A Method for Stochastic Optimization’, trong *Proceedings of the 3rd International Conference on Learning Representations (ICLR)*. San Diego, CA, 7-9 May.
- OpenCV Team** (n.d.). *OpenCV-Python Tutorials*. Có tại: https://docs.opencv.org/master/d6/d00/tutorial_py_root.html (Truy cập: 31/12/2025).
- Reshna, S., Haris, P.A., Imthias Ahamad, T.P. và Jayaraju, M.** (2025). ‘Hand Gesture Recognition for Indian Sign Language: A CNN-GRU-MLP Approach’. *International Journal of Intelligent Engineering & Systems*, 18(3), tr. 98–109.
- Soomro, K., Zamir, A.R. và Shah, M.** (2012). *UCF101: A Dataset of 101 Human Actions Classes From Videos in The Wild*. CRCV-TR-12-01. Orlando, FL: University of Central Florida.
- TensorFlow** (n.d.). *TensorFlow Core v2.x API Documentation*. Có tại: https://www.tensorflow.org/api_docs (Truy cập: 30/12/2025).
- Tran, D., Bourdev, L., Fergus, R., Torresani, L. và Paluri, M.** (2015). ‘Learning Spatiotemporal Features with 3D Convolutional Networks’, trong *Proceedings of the*

IEEE International Conference on Computer Vision (ICCV). Santiago, Chile, 11-18 December. IEEE, tr. 4489–4497.

UCF Center for Research in Computer Vision (2013). *UCF50 - Action Recognition Data Set*. Có tại: <https://www.crcv.ucf.edu/data/UCF50.php> (Truy cập: 01/01/2026).

Vrskova, R., Kamencay, P., Hudec, R. và Sykora, P. (2023). ‘A New Deep-Learning Method for Human Activity Recognition’. *Sensors*, 23(5), tr. 2816.

Zhao, J., Wang, G., Zhou, B., Ying, J. và Liu, J. (2023). ‘Exploring an application-oriented land-based hyperspectral target detection framework based on 3D–2D CNN and transfer learning’. *EURASIP Journal on Advances in Signal Processing*, 2023(1), tr. 16.

PHỤ LỤC

PHỤ LỤC A: CẤU HÌNH MÔI TRƯỜNG VÀ THƯ VIỆN THỰC NGHIỆM

1. Cấu hình phần cứng (Hardware Configuration)

Mô hình được huấn luyện và kiểm thử trên nền tảng đám mây (Kaggle/Colab) với cấu hình cụ thể như sau:

Nền tảng: Kaggle Notebooks / Google Colab.

Bộ xử lý đồ họa (Accelerator): GPU NVIDIA Tesla T4 / P100

Kiến trúc: x86_64.

2. Môi trường phần mềm (Software Environment)

Ngôn ngữ lập trình: Python 3.12

Các thư viện chính:

Tên thư viện	Mục đích sử dụng chính trong báo cáo	Ghi chú nguồn
TensorFlow / Keras	Xây dựng và huấn luyện mô hình 3D CNN (Conv3D, LSTM, v.v.)	tensorflow, tensorflow.keras
OpenCV (cv2)	Đọc video, xử lý ảnh, thay đổi kích thước khung hình	import cv2
Scikit-learn	Chia tập dữ liệu (train/test split), tính toán ma trận nhầm lẫn	sklearn.model_selection, sklearn.metrics
MoviePy	Xử lý file video cấp cao, hỗ trợ tiền xử lý	moviepy.editor
NumPy	Xử lý mảng đa chiều, tính toán số học	numpy (phiên bản ~2.0.2)
Matplotlib	Vẽ biểu đồ quá trình huấn luyện và ma trận nhầm lẫn	matplotlib.pyplot

3. Thiết lập hạt giống ngẫu nhiên (Random Seed)

Để đảm bảo tính tái lập (reproducibility) của kết quả thực nghiệm, hạt giống ngẫu nhiên được cố định:

Seed constant: SEED = 27

Áp dụng cho: np.random, random, và tf.random.

PHỤ LỤC B: CHI TIẾT VỀ TẬP DỮ LIỆU (DATASET)

1. Tổng quan về dữ liệu

Hệ thống sử dụng một tập con (subset) của bộ dữ liệu UCF50 (Action Recognition Data Set). Đây là bộ dữ liệu chuẩn bao gồm các video clip thực tế về các hành động, được thu thập từ YouTube.

Nguồn dữ liệu: UCF50.

Tổng số lớp hành động trong dataset gốc: 50 lớp.

Số lớp hành động được chọn để huấn luyện: 4 lớp.

2. Danh sách các lớp hành động (Class Names)

Mô hình được huấn luyện và đánh giá cụ thể trên 4 nhãn hành động sau:

- WalkingWithDog (Dắt chó đi dạo) - Nhãn 0
- TaiChi (Tập Thái Cực Quyền) - Nhãn 1
- Swing (Chơi xích đu) - Nhãn 2
- HorseRace (Đua ngựa) - Nhãn 3

3. Các bước tiền xử lý dữ liệu (Preprocessing Details)

Dữ liệu video thô được tiền xử lý để chuẩn hóa kích thước không gian và thời gian trước khi đưa vào mô hình 3D CNN. Các tham số cấu hình cụ thể như sau:

- Kích thước khung hình (Spatial Dimension): Tất cả các khung hình (frames) được thay đổi kích thước (resize) về độ phân giải cố định:
 - Chiều cao (Height): 64 pixels
 - Chiều rộng (Width): 64 pixels
- Độ dài chuỗi thời gian (Temporal Dimension): Mỗi video được chuẩn hóa thành một chuỗi khung hình có độ dài cố định để đảm bảo tính nhất quán cho đầu vào của mạng nơ-ron:
- Độ dài chuỗi (Sequence Length): 20 frames
- Phương pháp xử lý: Nếu video dài hơn 20 frames, hệ thống sẽ trích xuất 20 frames cách đều nhau (skip-frames) để bao quát toàn bộ hành động.

4. Phân chia tập dữ liệu (Dataset Splitting)

(Lưu ý: Bạn có thể bỏ sung biểu đồ hoặc bảng số lượng cụ thể từ output của lệnh `print(len(features_train))` trong notebook nếu đã chạy code)

Tỷ lệ phân chia (dự kiến theo cấu hình chuẩn): Dữ liệu được chia ngẫu nhiên thành các tập Huấn luyện (Train), Kiểm thử (Test) và Xác thực (Validation) để đánh giá độ chính xác và khả năng tổng quát hóa của mô hình.

Hạt giống ngẫu nhiên (Random Seed): 27 (đảm bảo việc xáo trộn và chia dữ liệu là cố định qua các lần chạy).

PHỤ LỤC C: CHI TIẾT KIẾN TRÚC MÔ HÌNH VÀ MÃ NGUỒN

1. Bảng tóm tắt kiến trúc mạng (Model Architecture)

Mô hình sử dụng kiến trúc 3D CNN (Convolutional Neural Network) thuần túy để trích xuất đặc trưng không gian (spatial) và thời gian (temporal) đồng thời.

Input Shape: (20, 64, 64, 3) – Tương ứng với (Số frames, Chiều cao, Chiều rộng, Số kênh màu RGB).

Output: 4 lớp hành động (Softmax).

Cấu trúc chi tiết các lớp:

Khối (Block)	Loại lớp (Layer Type)	Cấu hình tham số (Configuration)	Output Shape
Input	InputLayer	-	(None, 20, 64, 64, 3)
Block 1	Conv3D	32 filters, kernel (3,3,3), padding='same'	(None, 20, 64, 64, 32)
	MaxPooling3D	pool_size (1, 2, 2) (Chỉ giảm không gian)	(None, 20, 32, 32, 32)
	Dropout	rate = 0.25	-
Block 2	Conv3D	64 filters, kernel (3,3,3), padding='same'	(None, 20, 32, 32, 64)
	MaxPooling3D	pool_size (2, 2, 2)	(None, 10, 16, 16, 64)
	Dropout	rate = 0.3	-
Block 3	Conv3D	128 filters, kernel (3,3,3), padding='same'	(None, 10, 16, 16, 128)

	MaxPooling3D	pool_size (2, 2, 2)	(None, 5, 8, 8, 128)
	Dropout	rate = 0.4	-
Pooling	GlobalAveragePooling3D	-	(None, 128)
Dense	Dense	128 units, activation='relu'	(None, 128)
	Dropout	rate = 0.5	-
Output	Dense	NUM_CLASSES, activation='softmax'	(None, 4)

2. Các siêu tham số huấn luyện (Training Hyperparameters)

Các tham số này được thiết lập trước khi bắt đầu quá trình model.fit():

Số lượng Epochs: 100 (Số lần mô hình học toàn bộ tập dữ liệu).

Kích thước lô (Batch Size): 4 (Do xử lý video 3D tốn nhiều bộ nhớ, batch size nhỏ giúp ổn định quá trình học trên GPU T4).

Hàm mất mát (Loss Function): Sparse Categorical Crossentropy (Phù hợp cho bài toán phân loại đa lớp).

Thuật toán tối ưu (Optimizer): Adam (Adaptive Moment Estimation).

Cơ chế Callbacks:

EarlyStopping: Dừng sớm nếu val_loss không giảm sau 10 epochs (patience=10).

ReduceLROnPlateau: Giảm learning rate nếu val_loss không cải thiện sau 3 epochs (patience=3).

PHỤ LỤC D: HƯỚNG DẪN SỬ DỤNG VÀ CHẠY CHƯƠNG TRÌNH

1. Yêu cầu hệ thống và Môi trường thực thi

Để đảm bảo chương trình hoạt động ổn định và thời gian huấn luyện tối ưu, mã nguồn nên được thực thi trên các nền tảng đám mây hỗ trợ GPU miễn phí.

Nền tảng khuyên dùng: Google Colab hoặc Kaggle Notebooks.

Tài nguyên phần cứng:

GPU: Bắt buộc (Khuyến nghị NVIDIA Tesla T4 hoặc P100). Do mạng 3D CNN yêu cầu khối lượng tính toán lớn, việc chạy trên CPU sẽ rất chậm.

RAM: Tối thiểu 12GB.

Tệp tin nguồn: actionreg-with-3dcnn.ipynb.

2. Chuẩn bị dữ liệu (Dataset Preparation)

Hệ thống sử dụng bộ dữ liệu UCF50.

Bước 1: Tải bộ dữ liệu UCF50 từ nguồn chính thức hoặc Kaggle Dataset.

Bước 2: Giải nén tệp tin .zip hoặc .rar (nếu có).

Bước 3: (Đối với Google Colab):

Tải thư mục dữ liệu UCF50 lên Google Drive cá nhân.

Đảm bảo cấu trúc thư mục như sau:

- Thư mục gốc: My Drive
 - Thư mục UCF50, bao gồm các thư mục hành động:
 - WalkingWithDog
 - TaiChi
 - Swing
 - HorseRace
 - ... (các thư mục hành động khác trong bộ dữ liệu)

3. Các bước thực thi chương trình (Execution Steps)

Bước 1: Khởi động Notebook

Truy cập Google Colab (hoặc Kaggle).

Chọn File > Upload Notebook và tải lên file actionreg-with-3dcnn.ipynb.

Bước 2: Kích hoạt GPU

Trên thanh menu, chọn Runtime (Thời gian chạy) > Change runtime type (Thay đổi loại thời gian chạy).

Tại mục Hardware accelerator, chọn T4 GPU. Nhấn Save.

Bước 3: Cài đặt thư viện và Kết nối dữ liệu

Chạy cell đầu tiên để cài đặt các thư viện cần thiết (nếu môi trường chưa có sẵn).

(Nếu dùng Google Colab): Chạy cell kết nối Google Drive:

```
from google.colab import drive
drive.mount('/content/drive')
```

Bước 4: Cấu hình đường dẫn dữ liệu

Tìm đến cell có biến DATASET_DIR.

Sửa đổi đường dẫn trỏ đến thư mục chứa dataset bạn vừa chuẩn bị. Ví dụ:

```
DATASET_DIR = "/content/drive/MyDrive/UCF50"
```

Bước 5: Huấn luyện và Đánh giá

Chọn Runtime > Run all (Chạy tất cả) để hệ thống tự động thực hiện tuần tự các bước:

1. Trích xuất dữ liệu và Tiền xử lý (Feature Extraction).
2. Chia tập dữ liệu (Train/Test/Validation Split).
3. Xây dựng và Huấn luyện mô hình 3D CNN.
4. Vẽ biểu đồ và Xuất kết quả đánh giá.

4. Khắc phục sự cố thường gặp (Troubleshooting)

Hiện tượng	Nguyên nhân dự đoán	Giải pháp khắc phục
Lỗi <code>ResourceExhaustedError</code>	Tràn bộ nhớ VRAM GPU.	Giảm <code>BATCH_SIZE</code> xuống 2 hoặc 4.
Lỗi <code>FileNotFoundError</code>	Sai đường dẫn dataset.	Kiểm tra lại <code>DATASET_DIR</code> và lệnh <code>mount</code> .
Tốc độ train quá chậm ($>10p/epoch$)	Chưa kích hoạt GPU.	Kiểm tra lại cài đặt Runtime, chạy lệnh <code>!nvidia-smi</code> để xác nhận GPU đang hoạt động.